

TALLINNA TEHNIKAÜLIKOOL
Infotehnoloogia teaduskond

Daniel Erg 213407IACB
Hendrik Haabu 213440IACB
Kristjan Kaal 213423IACB

Targa laborikapi edasiarendus

Arvutite ja süsteemide projekt

Juhendaja: Heiko Pikner
PhD

Tallinn 2024

Autorideklaratsioon

Kinnitame, et oleme koostanud antud projekti iseseisvalt ning seda ei ole kellegi teise poolt varem kaitsmisele esitatud. Kõik töö koostamisel kasutatud teiste autorite tööd, olulised seisukohad, kirjandusallikatest ja mujalt pärinevad andmed on töös viidatud.

Autorid: Daniel Erg, Hendrik Haabu, Kristjan Kaal

10.12.2024

Lühendite ja mõistete sõnastik

RTOS	<i>Real-time operating system</i> , reaalaja operatsioonisüsteem
GPIO	<i>General Purpose Input Output</i> , üldkasutatav sisend-/väljundviik
JSON	<i>JavaScript Object Notation</i> , andmevahetusvorming
SPI	<i>Serial Peripheral Interface</i> , sünkroonse järjestiksuhtluse liidese standard
Wi-Fi	Sertifitseeritud IEEE 802.11 standardil põhinevad traadita kohtvõrgu klassi kuuluvad seadmed
NFC	<i>Near-field communication</i> , kontaktivaba lähiväljasidetehnoloogia
NO	<i>Normally open</i> , "tavapäraselt avatud" väljaviik
SPA	<i>Single-page application</i> , üheleherakendus
HTML	<i>Hypertext Markup Language</i> , veebilehtede märgenduskeel
CSS	<i>Cascading Style Sheets</i> , küljendamisel kasutatav märgistuskeel
URL	<i>Uniform resource locator</i> , internetiaadress
SSID	<i>Set Service Identifier</i> , võrgu nimi traadita võrgus

Sisukord

1 Sissejuhatus	8
2 Mehaanika	9
2.1 Prototüüp	9
2.1.1 Mooduli väline korpus.....	11
2.1.2 Mooduli kinnitus kapi uksele	12
2.1.3 Elektroonikakarp	13
2.1.4 Kapi avamise mehhanism.....	14
2.1.5 Kapi lukustamise mehhanism.....	17
2.2 Lõplik moodul	21
2.2.1 Mooduli välise korpuse muudatused	22
2.2.2 Mooduli uuendatud kinnitus kapi uksele.....	24
2.2.3 Elektroonikakarbi muudatused.....	25
2.2.4 Uuendatud kapi avamise mehhanism	26
2.2.5 Kapi lukustamise mehhanismi muudatused	28
3 Riistvara.....	29
3.1 Elektriskeem	30
4 Tarkvara.....	32
4.1 Andmemudel	32
4.1.1 Veebikasutajate haru <i>users</i>	33
4.1.2 Kapikasutajate haru <i>lockers</i>	34
4.2 Autentimine ja autoriseerimine	35
4.3 Veebirakendus	36
4.3.1 Komponendid <i>router.ts</i> ja <i>app.vue</i>	37
4.3.2 Navigatsiooniribade komponendid.....	38
4.3.3 Kapi seadete-, logide- ja kasutajahaldamisvaadete komponendid	39
4.3.4 Pea- ehk koduvaate komponendid.....	39
4.3.5 Administratiivvaate komponendid	40
4.4 Püsivara	43
4.4.1 Projekti seadistus	43

4.4.2 Platvormivahetusest tingitud muudatused	43
4.4.3 Vana kapiprojekti vigade parandus	44
4.4.4 Mitmetuumaline lahendus	44
5 Mooduli paigaldamise juhend	46
5.1 Kliendile antavad komponendid	46
5.1.1 Välimine korpus koos Gerd Kanteriga	46
5.1.2 Kinnitusdetailid	47
5.1.3 Elektroonikakarp	48
5.2 Mooduli paigaldamine kapile	49
6 Edasiarenduse võimalused	54
7 Kokkuvõte	55
Kasutatud kirjandus	55
Lisa 1 – Link koodihoidlale ja veebilehele	58
Lisa 2 – Reaalaja andmebaasi turvareeglid	59

Jooniste loetelu

Joonis 1. Nutikapi mooduli prototüüp.	10
Joonis 2. Nutikapi mooduli prototüüp paigaldatud esimesele laborikapi variandile.....	10
Joonis 3. Nutikapi mooduli prototüüp paigaldatud teisele laborikapi variandile.	11
Joonis 4. Mooduli välise korpuse detailid.	11
Joonis 5. Mooduli kinnituse detailid.	12
Joonis 6. Pikkvint Ekraanikorpuse silindri küljes.	13
Joonis 7. Elektroonikakarbi detailid.	13
Joonis 8. Kapi avamise eest vastutavad detailid.....	14
Joonis 9. Laborikapi hoovasüsteem.....	14
Joonis 10. Gerd Kanter Hoidiku sees ja fikseerimas Käepidet Ekraanikorpuse külge. .	15
Joonis 11. Hoovasüsteem Hoidiku ja Naagelvärgi vahel.	15
Joonis 12. Hoidik ja Naagelvärk altvaates.	16
Joonis 13. Kapi avamise liigutus.	16
Joonis 14. Nutikapi moodulit läbiv ava.	17
Joonis 15. Käepide koos postkastiluku komplektiga ning tugipunkt Elektroonikakarbis	18
Joonis 16. Postkastiluku keele kinnitus poldi otsas.....	19
Joonis 17. Solenoid Elektroonikakarbist ning Käepideme ja keele asend kui kapp on lukustatud.	20
Joonis 18. Postkastiluku keele liikumine üle solenoidi keele.....	20
Joonis 19. Keele keeramine võtmega 90° võrra vastupäeva.	21
Joonis 20. Targa laborikapi mooduli lõppversioon.	22
Joonis 21. Uue ja vana Ekraanikorpuse võrdlus eest ja tagant.	23
Joonis 22. Kimalane ja selle asetsemine Ekraanikorpuse süvendis.....	23
Joonis 23. Uus Käepideme mudel ja Toru.....	24
Joonis 24. Mooduli uuendatud kinnituse süsteem.	24
Joonis 25. Kimalase kinnitumine IKEA Laua külge.	25
Joonis 26. Elektroonikakarbi muudatused.....	26
Joonis 27. Kapi uuendatud avamise mehhanismi detailid.....	27

Joonis 28. Toru, Gerd Kanter ja Hoidik kokku ühendatuna.....	27
Joonis 29. Käepideme 90° liikumisulatuse funktsioon Gerd Kanteri ja Kimalase vahel.	28
Joonis 30. Postkastiluku mutri augu suurendus võrreldes prototüübiga.	28
Joonis 31. Laborikapi elektriskeem.	31
Joonis 32. JSON formaadis andmemudel.....	33
Joonis 33. Veebirakenduse komponentdiagramm.	37
Joonis 34. „BasicNavBar.vue“ administraatori vaates erinevate aknasuuruste puhul ...	38
Joonis 35. „NavBar.vue“ erinevate aknasuuruste puhul.....	38
Joonis 36. Settings.vue kapiseadetepaneel.....	39
Joonis 37. „Home.vue“ vaade nelja kapiga.	40
Joonis 38. AdminPage.vue tabelivaade.	42
Joonis 39. AdminPage.vue veebikasutaja lisamise vaade.....	42
Joonis 40. Ava Ekraanikorpusest tulevate juhtmete jaoks ning prototüüp koos juhtmega.	46
Joonis 41. Prototüübiks kasutuses olnud Käepide ja esimese valmis komponendi jaoks vajalikud detailid.	47
Joonis 42. Kinnitusvahendid targa laborikapi mooduli paigaldamiseks.	48
Joonis 43. Detailid kolmanda valmis komponendi jaoks ning pistikud Elektroonikakarbi küljel.	49
Joonis 44. Näidis laborikapist eemaldatavatest detailidest ja järele jääv ava.....	49
Joonis 45. Ühe testitud laborikapi jaoks vajaminevad Tuulerõuged mooduli stabiliseerimiseks.....	50
Joonis 46. Välimise korpuse kinnitamine laborikapi ukse külge.	51
Joonis 47. Hoidiku ja Naagelvärgi kinnitamine Gerd Kanteri külge.	51
Joonis 48. Käepideme ja keele asendid Elektroonikakarbist.....	52
Joonis 49. Lõplikult paigaldatud targa laborikapi moodul.	53

1 Sissejuhatus

Käesolev töö on edasiarendus targa laborikapi projektist [1]. Tallinna Tehnikaülikoolis on laialdaselt kasutusel erinevat tüüpi laborikapid ja nii mõnelegi on soovitud paigaldada tark süsteem. Kuna NFC-l põhinevate lukusüsteemide kasutamise populaarsus on ajaga suurenenud, siis on teema aktuaalne [2].

Projekti ülesanne on arendada edasi varem tehtud tarka laborikappi. Eelneva töö puudusteks olid tarkvaras mitme kapi toe puudumine ja lahenduse realiseerimine mitme moodulina ehk mitme füüsilise tükina, mis muutis paigaldamise tülikaks, ja kapi originaalse riivi eemaldamise vajadus. Eelneva targa laborikapiga oli ka see probleem, et ühel hetkel seda ei saanud enam kaardiga avada ja mure oli ka see, et kui vool ära kadus, siis oli kapi avamine samuti üsna ebamugav. Seega antud töö eesmärk on ehitada võimalikult kompaktnel, soodne, universaalne ja lihtsasti paigaldatav moodul, mis muudaks metallist laborikapi targaks, kusjuures tarkvara peab võimaldama mitme kapi kasutust. Lahendus peab kasutama ka kapi originaalset riivi ja voolu kadumisel peab kapp olema avatav ka võtmega. Eesmärk on muuta ka kapp jälle kaardiga avatavaks.

Töö hõlmab endas nii riistvara, tarkvara, mehaanikat kui ka modelleerimist. Terve projekti vältel tegeleti nende kõikide mainitud osadega pidevalt, kuid alguses oli suurem rõhk modelleerimisel ja riistvaral. Töö peatükid on jaotatud samasugusel põhimõttel: alguses selgitatakse lukusüsteemi kogu mehaanika ja modelleerimisega seonduvat poolt, siis selgitatakse riistvara kasutust ja elektriskeemi ning lõpuks mõtestatakse lahti tarkvaraline osa, sealhulgas püsivara ja veebirakendus. Viimasteks peatükkideks on mooduli paigaldamise juhend ja edasiarenduse võimalused.

2 Mehaanika

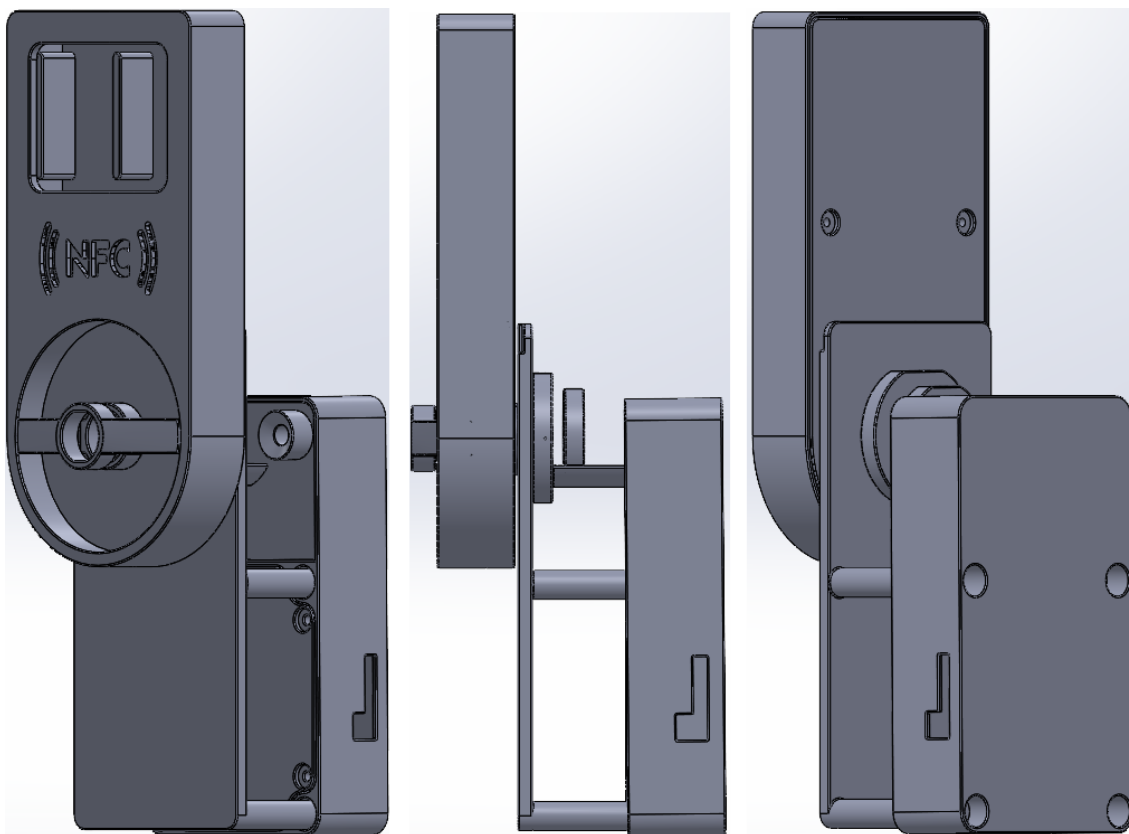
Nutikapi mooduli loomiseks modelleeriti esialgu prototüüp ning hiljem lõplik toode. Prototüüp prinditi plastikust ning seda kasutati kahel erineval laborikapil testimiseks. Katsete käigus tuvastatud puudujääkide tõttu tuli muuta nii detailide mõõtmeid kui ka modelleerida osad mudelid ümber eesmärgiga need hiljem metallist toota, sest need ei pidanud kapi hoovamehhanismi liigutades pingele vastu. Prototüübi peatükk kirjeldab esialgse mooduli jaoks ehitatud detaile, nende probleeme ja funktsionaalsust ning lõpliku mooduli peatükis on välja toodud muudatused toote viimase versioonini jõudmiseks. Kõik detailid on modelleeritud 3D projekteerimistarkvaras Solidworks.

2.1 Prototüüp

Prototüübi jaoks modelleeriti 9 detaili, mille nimed on Ekraanikorpus, Ekraanikorpuse Kaas, Käepide, IKEA Laud, Gerd Kanter, Hoidik, Naagelvärk, Elektroonikakarp ja Elektroonikakarbi Kaas.

Lisaks soetati ehitusmaterjali poodidest M1x10 mm, M3x16 mm ja M4x90 mm suurusega polte, 3.5x9.5 mm suurusega kruvisid, 3/4 tolli suurusega kontramutter ja pikkvint, nelinurkne M4 mutter, M4 suurusega võllikrae ja postkastiluku komplekt.

Joonisel 1 on näha prototüübi modelleeritud detailid kokkupanduna ning joonistel 2 ja 3 on näha prototüübi paigaldust kahele laborikapi variandile.



Joonis 1. Nutikapi mooduli prototüüp.



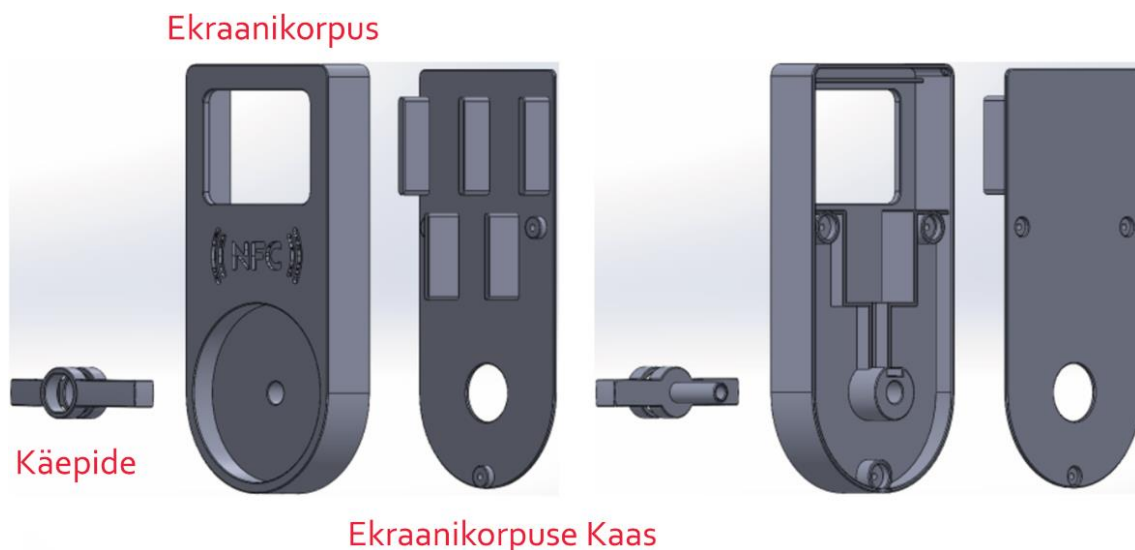
Joonis 2. Nutikapi mooduli prototüüp paigaldatud esimesele laborikapi variandile.



Joonis 3. Nutikapi mooduli prototüüp paigaldatud teisele laborikapi variandile.

2.1.1 Mooduli väline korpus

Mooduli väline korpus on laborikapist väljapoole jääv ekraani ja NFC moodulit hoiustav komponent (vt Joonis 2 ja 3, vasakpoolne pilt). See koosneb modelleeritud Käepidemest, Ekraanikorpusest ja Ekraanikorpuse Kaanest ning ehitusmaterjalide poest soetatud postkastilukust, kolmest 3.5x9.5 mm suurusega kruvist ja 3/4 tolli suurusega pikkvindist. Joonisel 4 on näha välise korpuse mudeleid nii eest kui ka tagant.



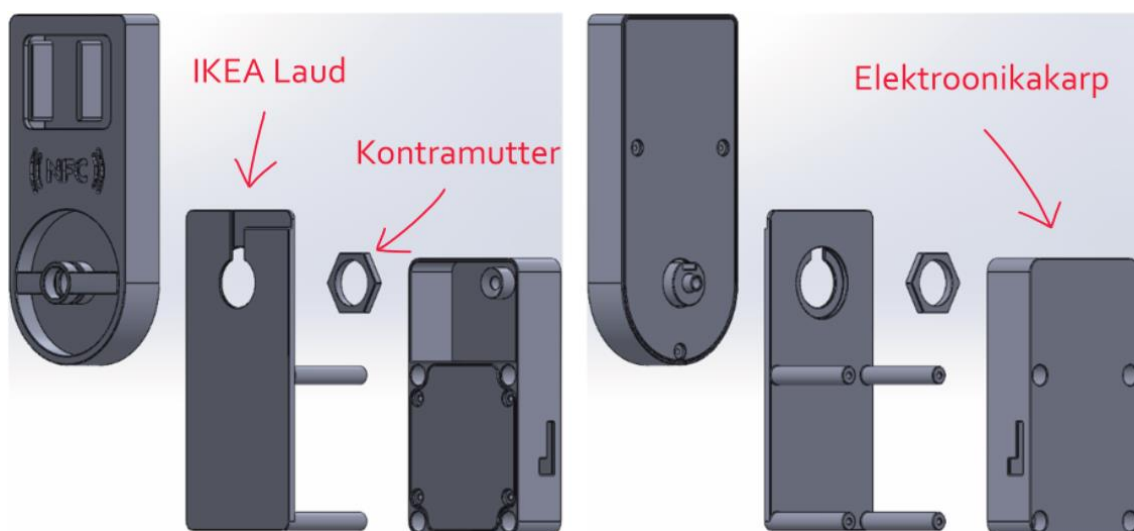
Joonis 4. Mooduli välise korpuse detailid.

Ekraanikorpuse sees olev suurem ristkülikukujuline süvend on mõeldud puutetundliku ekraani jaoks ning väiksem NFC mooduli jaoks. Korpuse kaanel on viis väljaulatuvat risttahukat, mille eesmärgiks on hoida ekraani ja NFC moodulit korpuse vastas. Kaanel

on ka kolm silindrikujulist jalga, mis sobituvad korpusel olevatesse analoogsetesse süvenditesse ning need kinnitatakse väljastpoolt kolme 3.5x9.5 mm suuruse kruviga. Joonisel 4 vasakult teisel detailil ehk Ekraanikorpusel on ka näha ringikujulist süvendit, mis on mõeldud Käepideme pöörlemiseks ning NFC logo, mille mudel on võetud eelmise laborikapi mooduli versiooni tegijatelt [1]. Käepideme toru ulatub läbi Ekraanikorpuse kapi sisse.

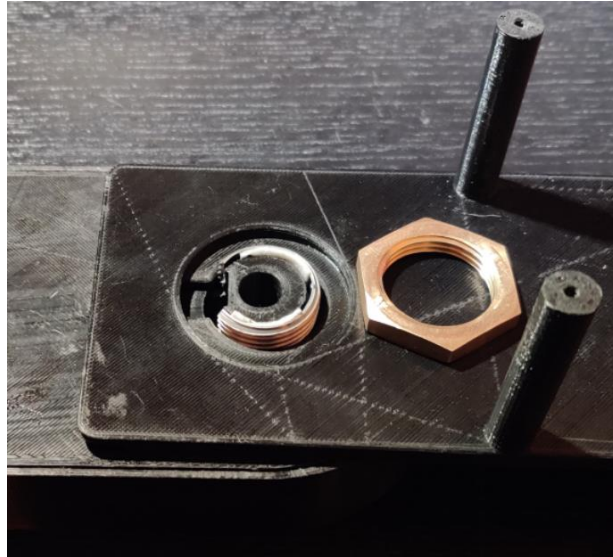
2.1.2 Mooduli kinnitus kapi uksele

Mooduli kinnitus kapi uksele on kapist sissepoole jääv komponent, mille eesmärk on hoida Ekraanikorpust ja Elektroonikakarpi kapi ukse küljes. See koosneb modelleeritud IKEA Lauast ning selle juures mängib olulist rolli ka Ekraanikorpuse tagant väljaulatav jäme silinder. Lisaks on kasutusel 3/4 tolline kontramutter ja pikkvint ning neli M3x16 mm polti. Joonisel 5 on näha kinnituse modelleeritud detailid.



Joonis 5. Mooduli kinnituse detailid.

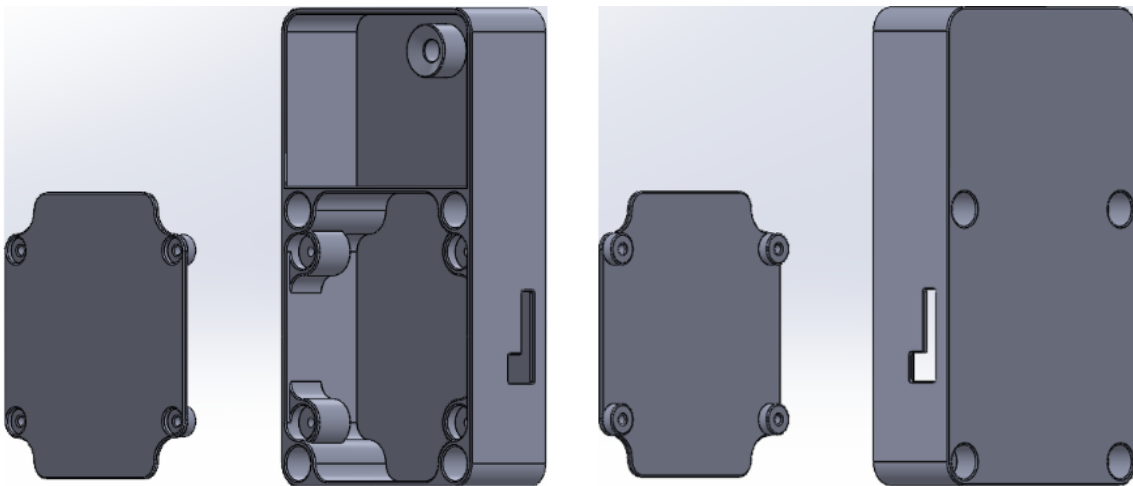
IKEA Laua jalgade külge kinnitub Elektroonikakarp ning seda hoiavad jalgade küljes neli M3x16 mm polti. Laud jääb Ekraanikorpusest allapoole, kuna mõlemal testitud laborikapil jäi sisemine riiul muidu Elektroonikakarbile ette. Ekraanikorpuse ja IKEA Laua vahele jääb laborikapi uks. Eemaldades ukse originaalse käepideme, jääb selle asemele ava ning läbi ava ja IKEA Laual oleva suure ümmarguse augu paigaldatakse Ekraanikorpust tänu tema küljes olevale jämedale silindrile. Prototüübi testimiseks saeti 3/4 tolline pikkvint sobivaks suuruseks ning kinnitati silindri ümber, mistõttu sai kontramutrit abil suruda IKEA Laua ja Ekraanikorpuse tugevalt ukse külge (vt Joonis 6).



Joonis 6. Pikkvint Ekraanikorpuse silindri küljes.

2.1.3 Elektroonikakarp

Elektroonikakarp on IKEA Laua külge kinnituv solenoidi ja elektriskeemi hoiustav komponent, mis koosneb modelleeritud karbist ja selle kaanest (vt Joonis 7). Kaane kinnitamiseks on kasutusel neli 3.5x9.5 mm kruvi.

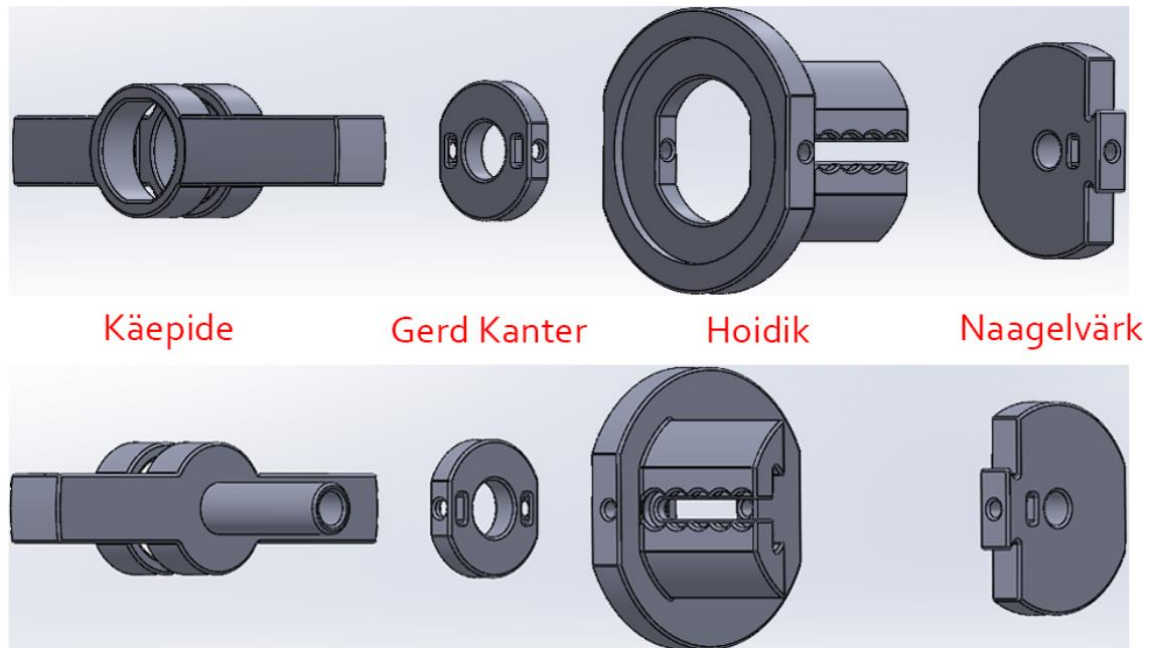


Joonis 7. Elektroonikakarbi detailid.

Elektroonikakarbi külje peal on avad 12 V toitepistiku ning ekraani ja NFC mooduli küljest tuleva 15 viiguga juhtme pistiku jaoks. Karbi üleval servas oleva silindri ning solenoidi täpsema asetuse ja eesmärgi kohta on kirjas kapi lukustamise mehhanismi alapeatükis.

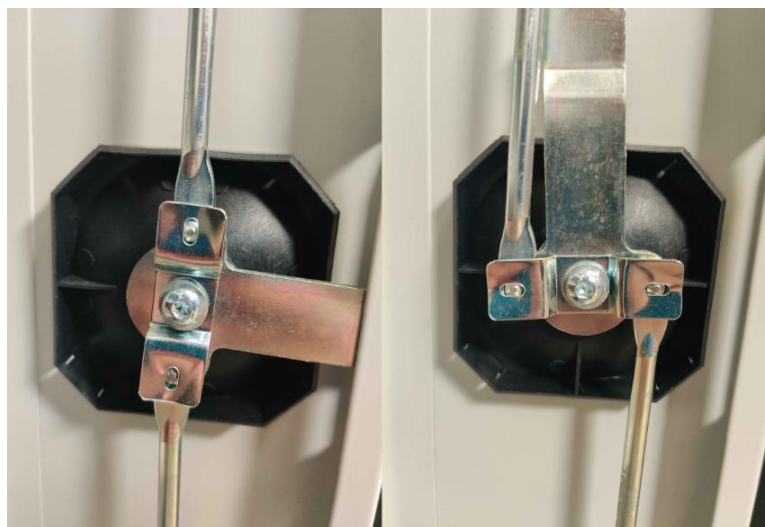
2.1.4 Kapi avamise mehhanism

Kapi avamise mehhanism on kapi hoovasüsteemi liigutav komponent. See koosneb modelleeritud Käepidemest, Gerd Kanterist, Hoidikust ja Naagelvärgist. Lisaks on kasutusel kolm M3 suurusega mutrit, kolm M3x16 mm polti ja kaks M1x10 mm suurusega polti. Joonisel 8 on näha kapi avamise mehhanismi jaoks modelleeritud detaile.



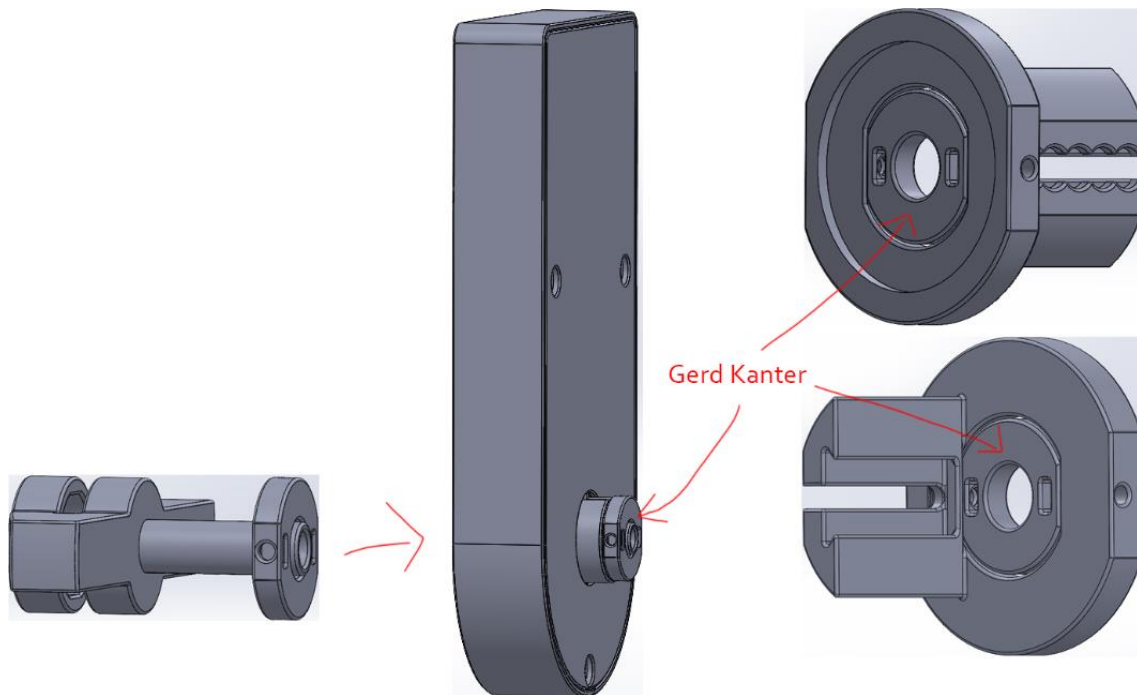
Joonis 8. Kapi avamise eest vastutavad detailid.

Need komponendid on loodud selliste laborikappide jaoks, mis avanevad joonisel 9 näha oleva hoovasüsteemi abil.



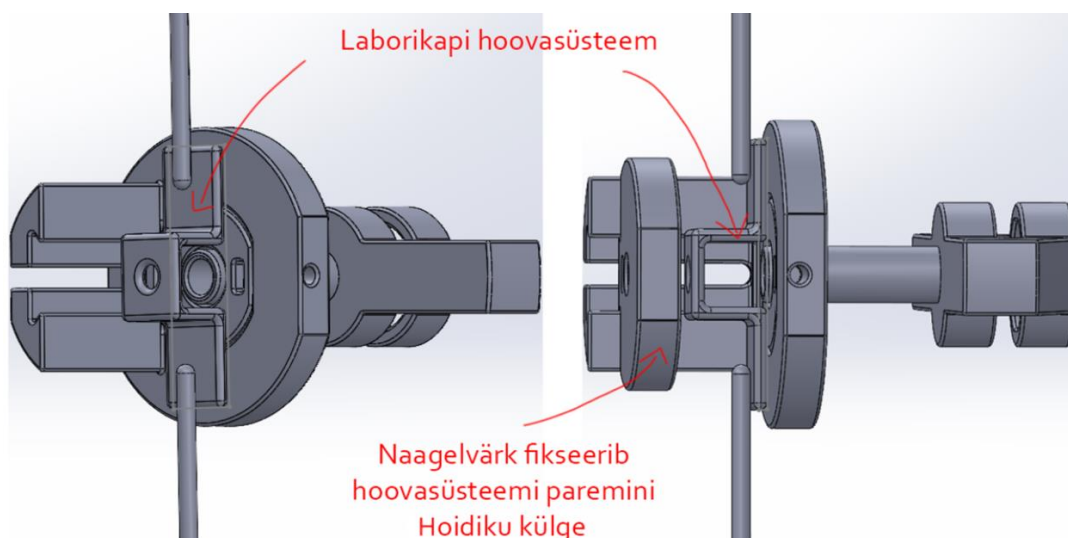
Joonis 9. Laborikapi hoovasüsteem.

Gerd Kanter kinnitub kahe M1x10 mm suuruse poldiga Käepideme toru külge ja kahe M3x16 mm poldiga Hoidiku külge. Tema küljes olevate piklike avade sisse kinnitati eraldi M3 mutrid, mis olid mõeldud ajutiste keermetena. Gerd Kanteri eesmärk on keerata Hoidikut ning vältida Käepideme välja kukkumist Ekraanikorpusest (vt Joonis 10).



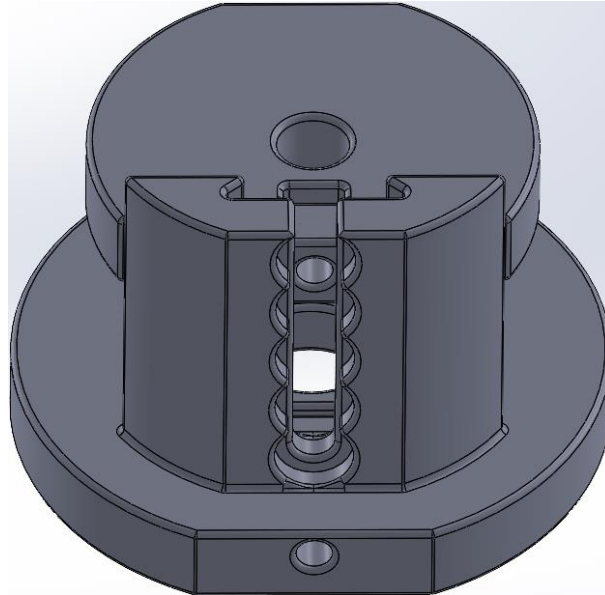
Joonis 10. Gerd Kanter Hoidiku sees ja fikseerimas Käepidet Ekraanikorpuse külge.

Hoidiku eesmärk on keerata laborikapi hoovasüsteemi ning sellele kinnitati Naagelvärk ühe pooleks saetud M3x16 mm poldiga hoovasüsteemi paremaks fikseerimiseks (vt Joonis 11).



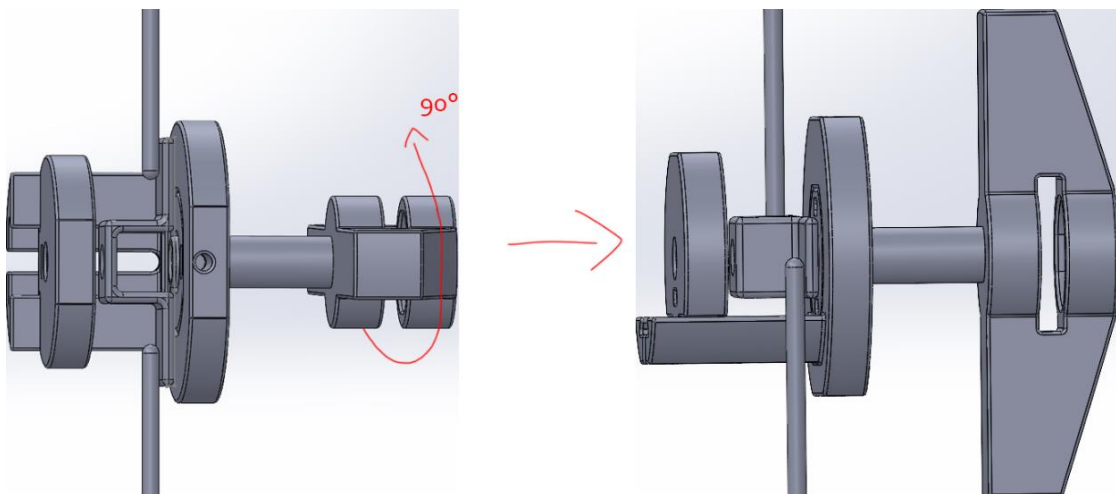
Joonis 11. Hoovasüsteem Hoidiku ja Naagelvärki vahel.

Hoidiku all on viis erinevat süvendit M3x16 mm poldi kinnitamiseks. Nende abil on võimalik Naagelvärki kaugust kapi hoovasüsteemist reguleerida, kuna erinevatel laborikappidel võib süsteem olla kas teistsuguse kujuga või asetseda kapi seinast kaugemal või lähemal. Ka Naagelvärkile oli prototüübi jaoks modelleeritud piklik auk ajutise mutri kinnitamiseks. Hoidikut ja Naagelvärki altvaates on näha joonisel 12.



Joonis 12. Hoidik ja Naagelvärk altvaates.

Kinnises asendis toimivad kapi hoovasüsteemist välja ulatuvad pikad vardad riividena ning hoiavad ust kapi ülemise ja alumise serva küljes kinni. Keerates käepidet 90° päripäeva, liigutab Hoidik hoovasüsteemi lahtisesse asendisse ning kapp avaneb. Hoovasüsteem kinnises asendis on näha joonisel 13 vasakul pool ja lahtises asendis paremal pool.



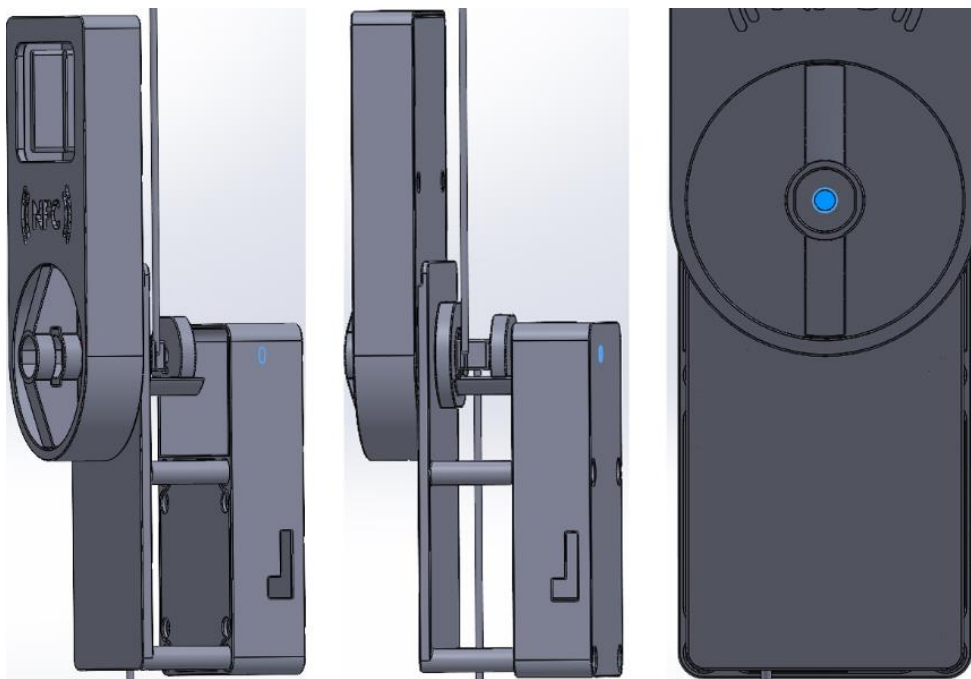
Joonis 13. Kapi avamise liigutus.

Kapi sulgemiseks tuleb keerata käepidet tagasi 90° vastupäeva.

2.1.5 Kapi lukustamise mehhanism

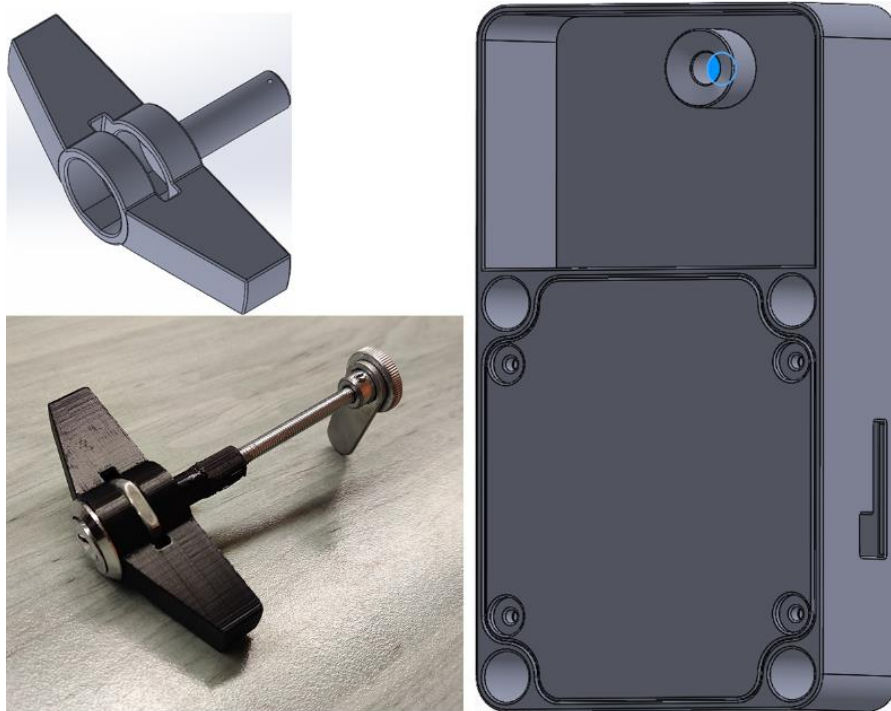
Kapi lukustamise mehhanism on kapi ukse lukustamise loogikat käsitlev komponent. Lisaks avamise mehhanismi komponendile, koosneb lukustamise komponent postkastiluku komplektist, ühest M4x90 mm poldist ning võllikraest, nelinurksest mutrist ja seibist, mis on kõik M4 suurusega. Riistvarast on kapi lukustamisel oluline roll solenoidil ja NFC moodulil.

Pannes kõik siamaani käsitletud komponendid tagasi kokku, on märgata, et läbi kogu mooduli on jätud 6 mm diameetriga ava (vt Joonis 14, sinine täpp).



Joonis 14. Nutikapi moodulit läbiv ava.

Kogu ava läbib M4x90 mm polt, mis on ühe otsaga kinnitatud Käepideme sees oleva postkastiluku külge ja teine ots toetub Elektroonikakarbi üleval servas oleva jämeda silindri sisse (vt Joonis 15).



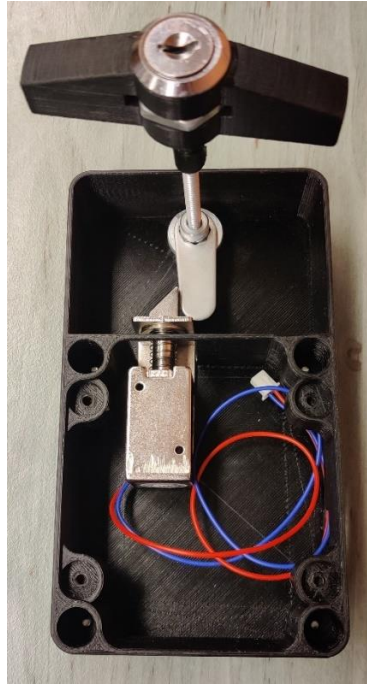
Joonis 15. Käepide koos postkastiluku komplektiga ning tugipunkt Elektroonikakarbis

Poldil on Elektroonikakarbi poolisel otsal keermed maha lihvitud ning teisel pool pea ära lõigatud. Lihvitud otsale puuriti tilluke süvend, millesse kinnitati esimesena väikse kruviga võllikrae ja teiseks postkastiluku keel, mille sees on sobivaks suuruseks lihvitud nelinurkne mutter eesmärgiga kanda jõudu Käepidemelt keelele. Järgmisena kinnitati poldile seib, mis hoiab keelt võllikrae vastas ning lõpus liimiti detailid tugeva kiirliimiga kokku (vt Joonis 16).



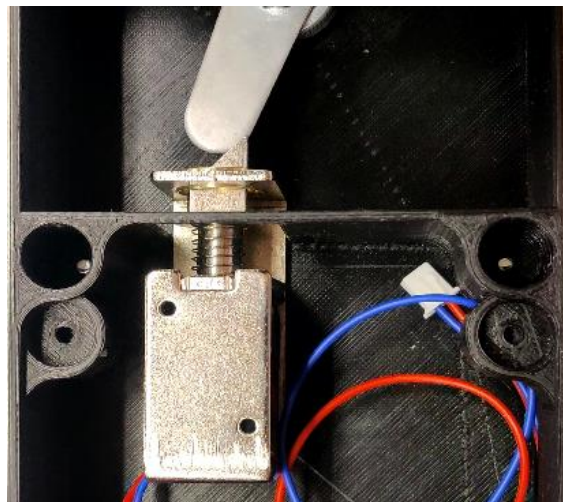
Joonis 16. Postkastiluku keele kinnitus poldi otsas.

Kuna postkastiluku keel on Käepideme küljes, ongi tema eesmärgiks lukustada kapp ehk blokeerida Käepideme liikumine. Keele liikumise takistamiseks on kasutusel solenoid. Joonisel 17 on näha solenoidi asukoht Elektroonikakarbis ning Käepideme ja keele asend kui kapp on lukustatud.



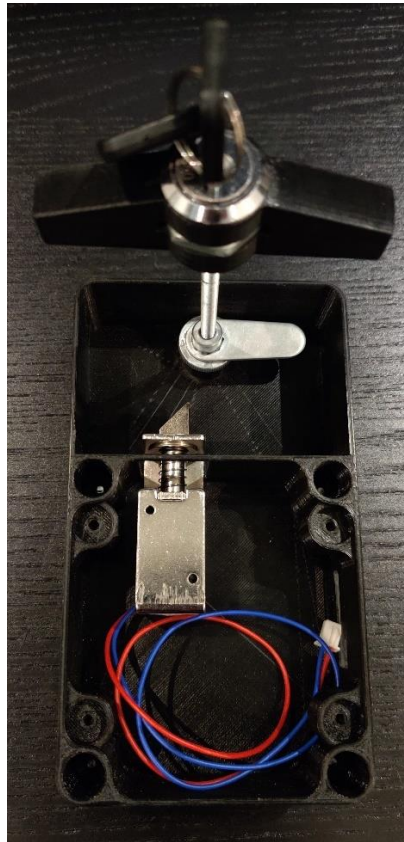
Joonis 17. Solenoid Elektroonikakabis ning Käepideme ja keele asend kui kapp on lukustatud.

Kuna kapi hoovasüsteemi liigutamiseks avatud asendisse tuleb käepide keerata 90° päripäeva, on see hetkel piiratud, sest solenoid takistab keele liikumist. Selleks, et Käepide liigutada, on esimeseks variandiks asetada ISIC kaart NFC mooduli vastu, mistõttu läheb solenoidi keel alla ja Käepide on võimalik keerata. Prototüübil on võimalik keerata ka Käepide täisnurga võrra vastupäeva, kuid lõplikul tootel on see takistatud. Kapi lukustamiseks tuleb Käepide keerata tagasi 90° vastupäeva ning postkastiluku keel liigub sujuvalt üle solenoidi keele ja jõuab tagasi suletud asendisse (vt Joonis 18).



Joonis 18. Postkastiluku keele liikumine üle solenoidi keele.

Teine variant kapi avamiseks on keerata võtmega postkastilukku. Sellisel juhul pöörleb keel 90° vastupäeva, mistõttu on võimalik taaskord Käepidet keerata täisnurga võrra päripäeva (vt Joonis 19).



Joonis 19. Keele keeramine võtmega 90° võrra vastupäeva.

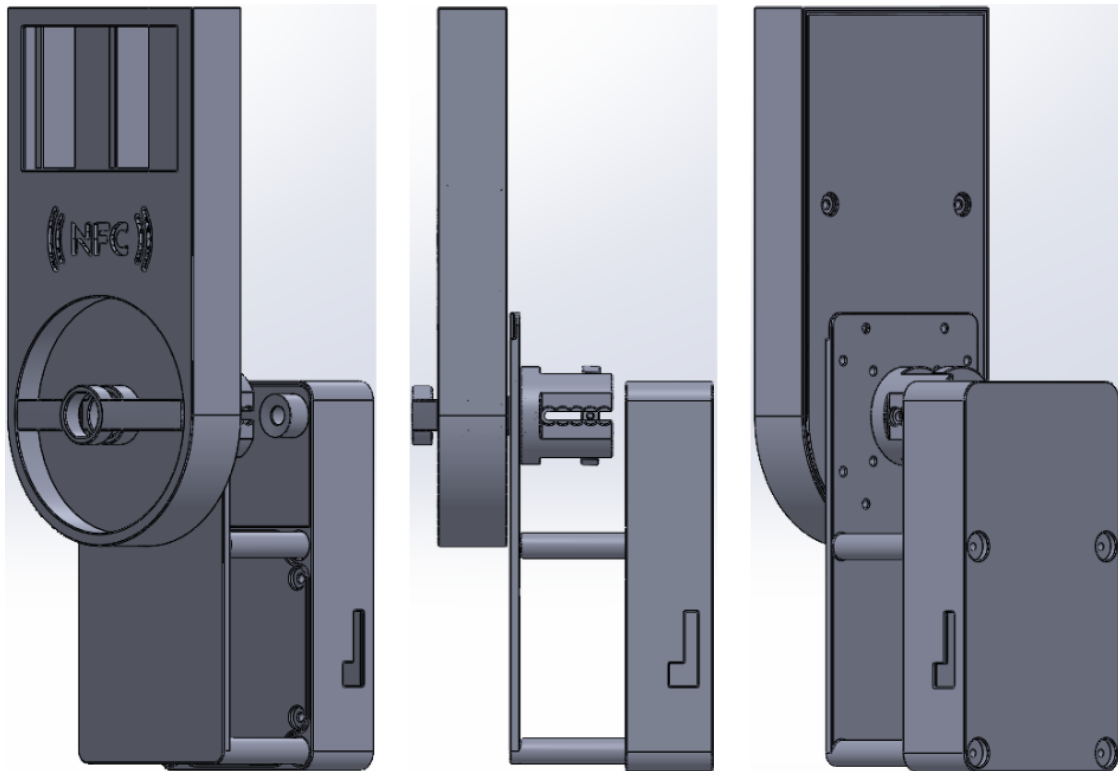
Kuna Käepidemest väljaulatuv polt pole kuidagi ühendatud Gerd Kanteri, Hoidiku ega hoovasüsteemiga ning saab neid läbides vabalt pöörelda, siis võtmega poldi ja keele keeramine ei liiguta Hoidikut ega Käepidet.

2.2 Lõplik moodul

Prototüübi testimisel laborikappidel oli põhiliseks probleemiks plastikdetailide nõrkus – hoovasüsteemi liigutavad ja moodulit kapi küljes hoidvad komponendid läksid mõõduka pinge all katki. Lisaks tuli muuta mõnde komponenti mõõtmeid. Seetõttu otsustati modelleerida osad detailid eesmärgiga need lõpuks metallist toota ning mõelda välja viisid, kuidas need plastikdetailide külge kinnitada. Kuna puudus metalliprinter, ei olnud võimalik uusi komponente toota, mistõttu neid ei testitud kordagi laborikappidel.

Lõplik toode koosneb 12 modelleeritud detailist nimedega Ekraanikorpuse, Ekraanikorpuse Kaas, Kimalane, Käepide, Toru, IKEA Laud, Mutter, Gerd Kanter, Hoidik, Naagelvärk, Elektroonikakarp ja Elektroonikakarbi Kaas.

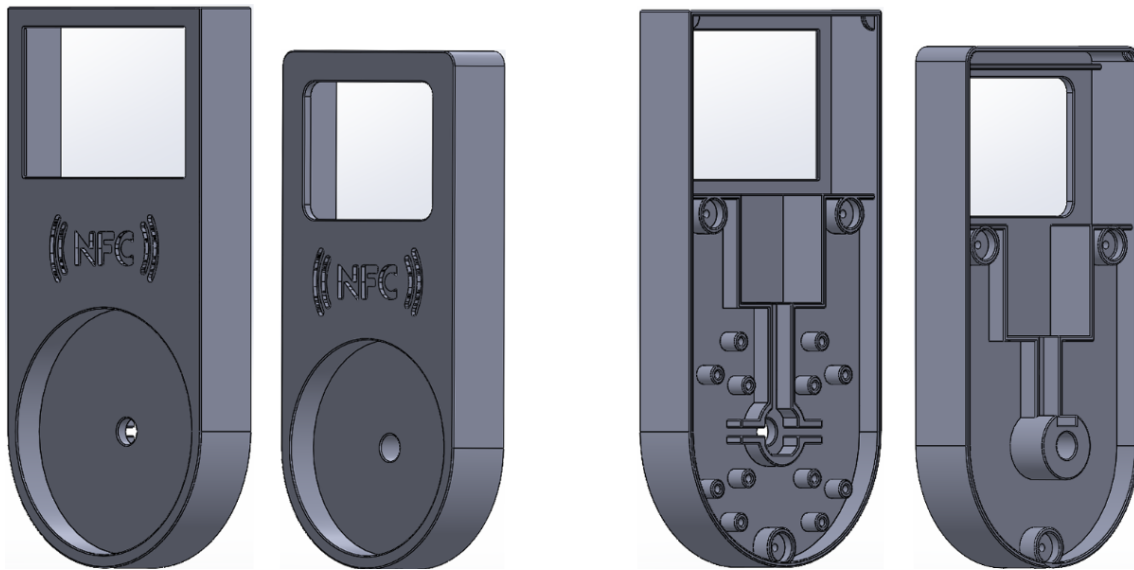
Kimalane, Toru, Mutter, Gerd Kanter, Hoidik ja Naagelvärk on mõeldud tootmiseks metallist, ülejäänud plastikust.



Joonis 20. Targa laborikapi mooduli lõppversioon.

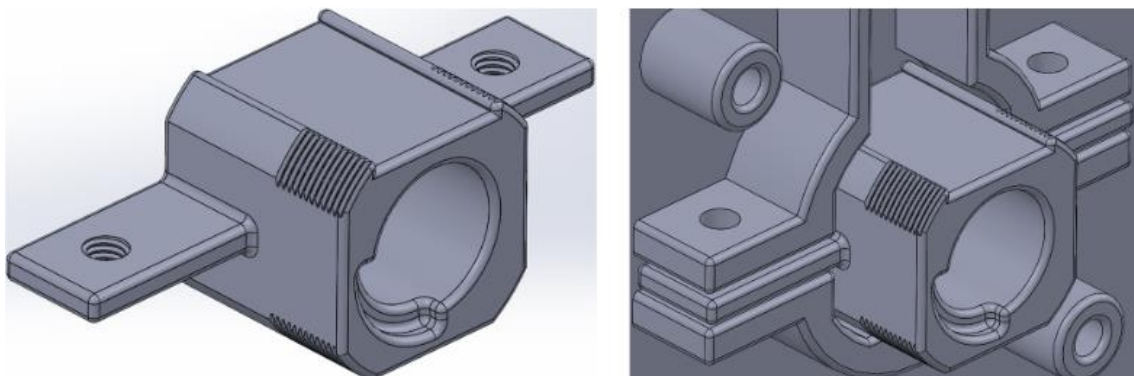
2.2.1 Mooduli välise korpuse muudatused

Uue mooduli väline korpuse koosneb lisaks Käepidemele, Ekraanikorpusele ja Ekraanikorpuse Kaanele ka Kimalasest ning Torust. Kimalane on metallist detail, mis asendab prototüübil olnud jämeda silindri ning eraldi pikkvindist saetud keermed. Toru on metallist detail, mis asendab eelneva Käepideme plastikust toru. Lisaks muudeti ekraaniava suurust, kuna vahepeal võeti kasutusele uus ekraanimoodul, ning korpusele lisati juurde Tuulerõuget ehk üheksa M3 poldi süvendit, millest on täpsemalt kirjas mooduli kapile paigaldamise peatükis. Joonisel 21 on näha paarides vastavalt uut ja vana korpust.



Joonis 21. Uue ja vana Ekraanikorpuse võrdlus eest ja tagant.

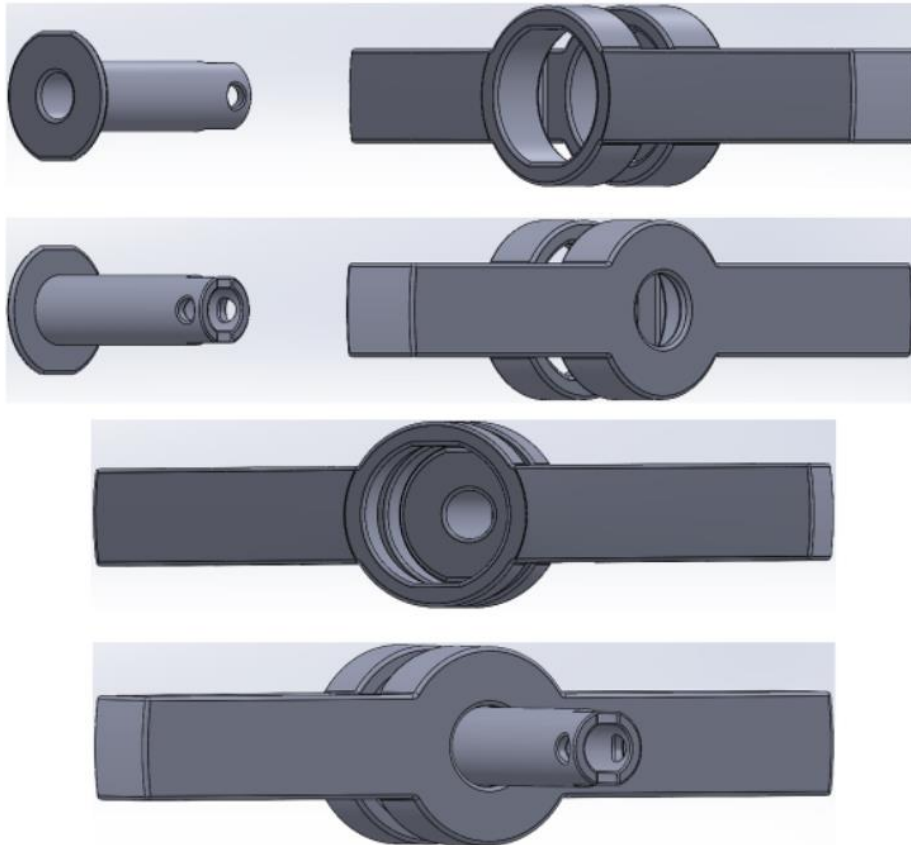
Kuna uus ekraan oli pikema diagonaaliga, muudeti kogu korpust suuremaks. Lisaks on tagant näha, kuidas jäme silinder on asendatud uue Kimalase pesaga. Kimalast ning selle asetsemist süvendis on näha joonisel 22.



Joonis 22. Kimalane ja selle asetsemine Ekraanikorpuse süvendis.

Kimalane kinnitub süvendisse kahe M3x16 mm poldiga ning tema tööpõhimõte on täpsemalt lahti seletatud mooduli uuendatud kinnituse ja uuendatud kapi avamise mehhanismi alapeatükkides.

Kuna eelmine plastikust Käepideme toru läks esimesena prototüübi testimisel pooleks, asendati see eraldi metallist detailiga, mida on koos uue Käepideme mudeliga näha joonisel 23.

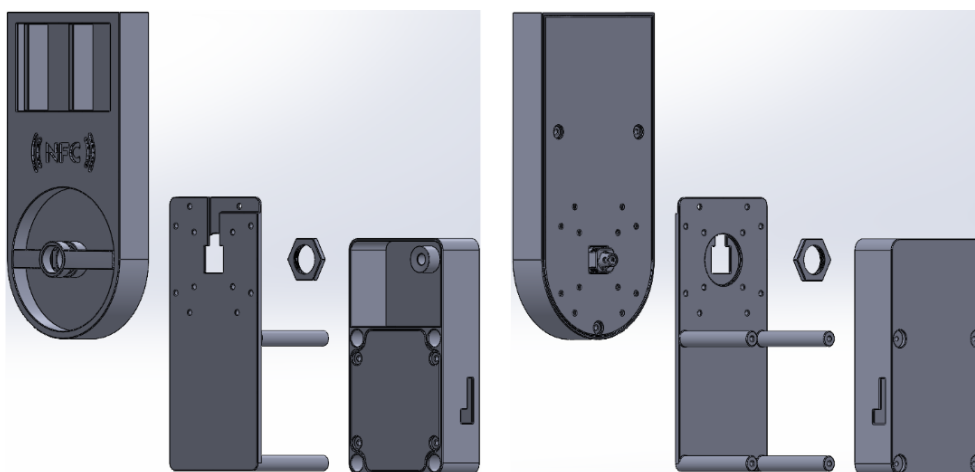


Joonis 23. Uus Käpideme mudel ja Toru.

Kuna Käpideme suurde süvendisse käib postkastilukk, surub see Toru kindlalt vastu Käpideme põhja ning Toru sirged servad takistavad sel kappi avades keerelda.

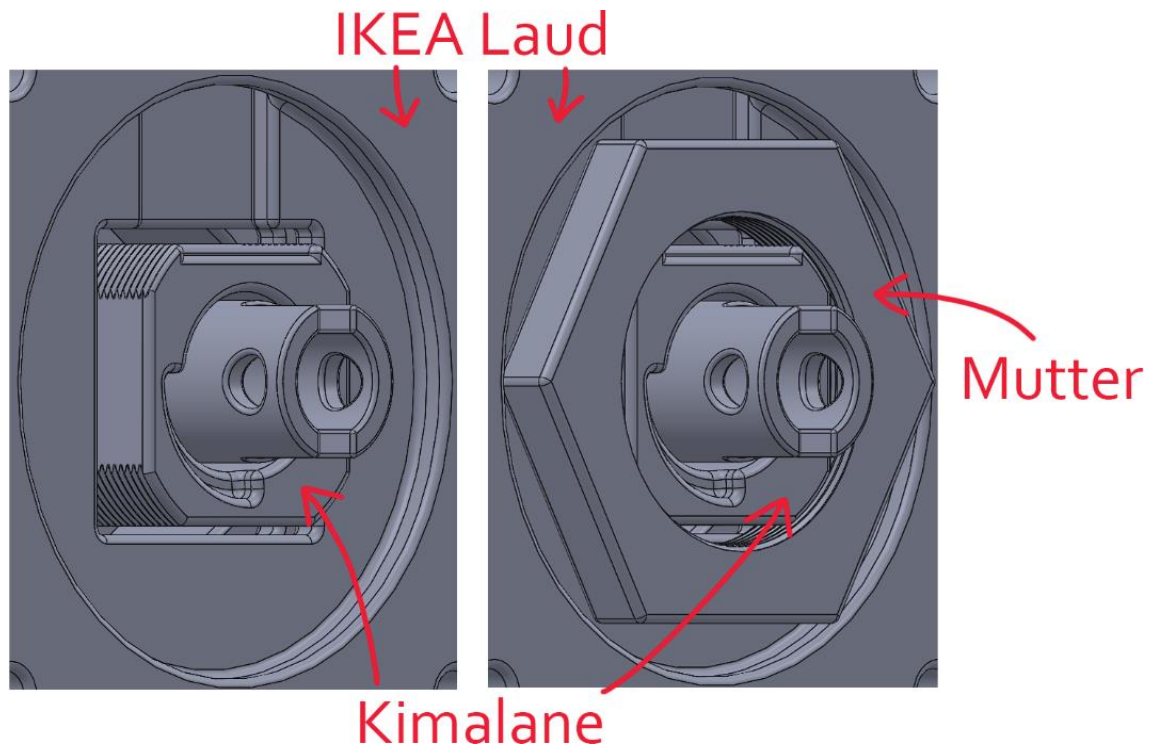
2.2.2 Mooduli uuendatud kinnitus kapi uksele

Mooduli uuendatud kinnituse eest vastutavad muudetud IKEA Laud, Kimalane ja uus modelleeritud Mutter. Joonisel 24 on näha uut kinnituse süsteemi.



Joonis 24. Mooduli uuendatud kinnituse süsteem.

Peamine erinevus vana süsteemiga on IKEA Lauale lisandunud Tuulerõuged, mis on ka Ekraanikorpuse Kaanel, ning Laua suure ava kuju muutmine ristkülikukujuliseks. Kuna vana Ekraanikorpuse jämedat silindrit asendab nüüd Kimalane, mis on samuti ruudukujuline, püsib mooduli kapisisene osa alati välise korpusega paralleelselt. Lähemat vaadet Kimalase kinnitumisest IKEA Laua külge on näha joonisel 25.

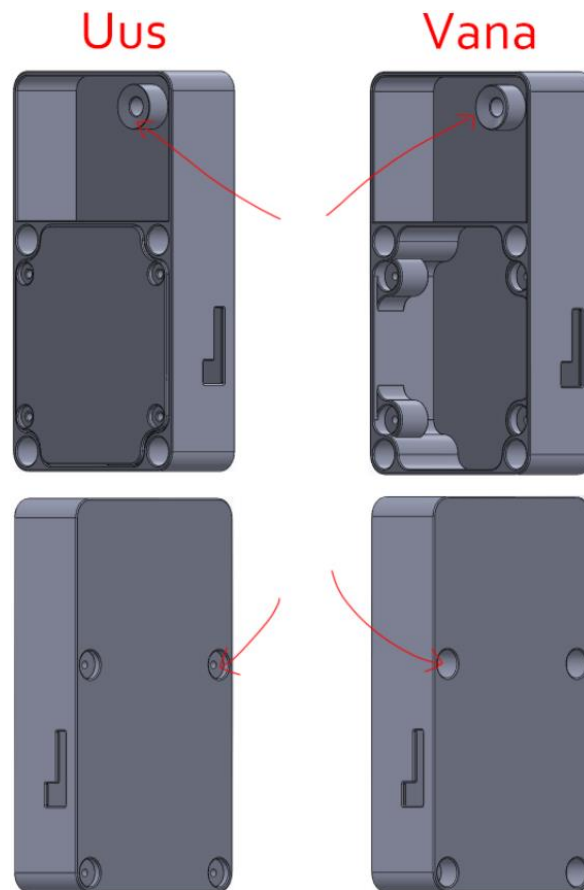


Joonis 25. Kimalase kinnitumine IKEA Laua külge.

Lisaks modelleeriti juurde spetsiaalselt nutikapi moodulile sobiv Mutter, kuna ehitusmaterjalide poest sobivat mutrit ei leitud.

2.2.3 Elektroonikakarbi muudatused

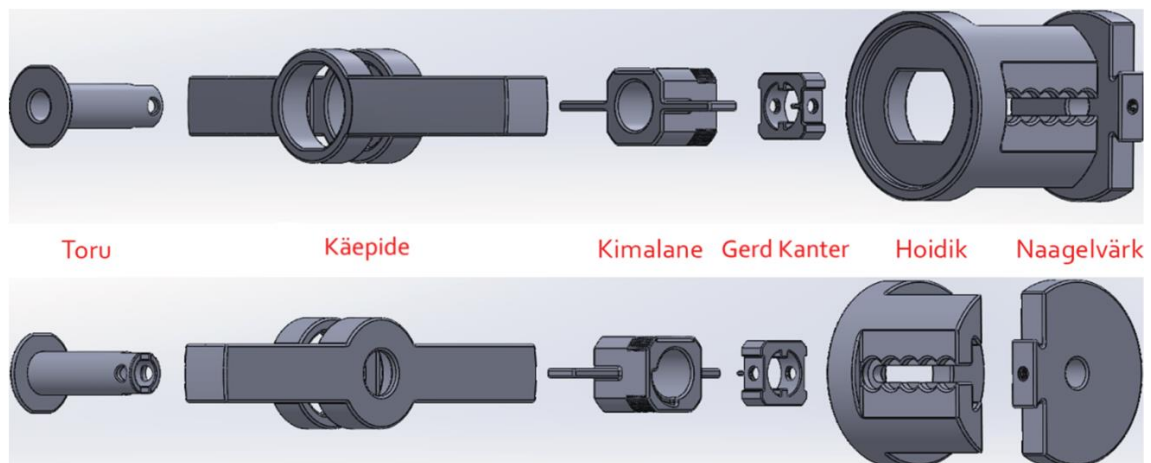
Elektroonikakarpi muudeti lõplikus moodulis võrreldes teiste komponentidega väga vähe. Vähendati karbi taga olevate kruvisüvendite sügavust ning tehti postkastiluku poldi tugipunkti silindrit sügavamaks. Vana ja uue elektroonikakarbi erinevused on näha joonisel 26.



Joonis 26. Elektroonikakarbi muudatused.

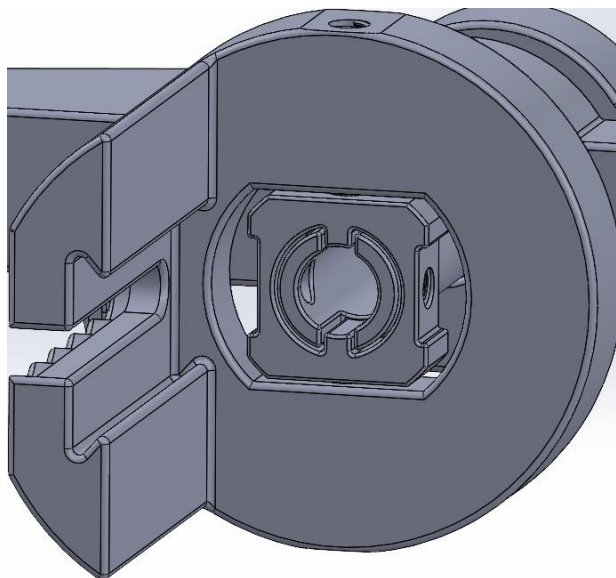
2.2.4 Uuendatud kapi avamise mehhanism

Enamik muudatusi võrreldes prototüübi detailidega tehti kapi avamise mehhanismi juures. Uus mehhanism koosneb Käepidemest, Torust, Kimalasest, Gerd Kanterist, Hoidikust ja Naagelvärgist. Esiteks on iga mudel nüüd mõeldud tootmiseks metallist, teiseks muudeti iga detaili kuju ja suurust ning kolmandaks mängib olulist rolli kapi avamisel ka Kimalane.



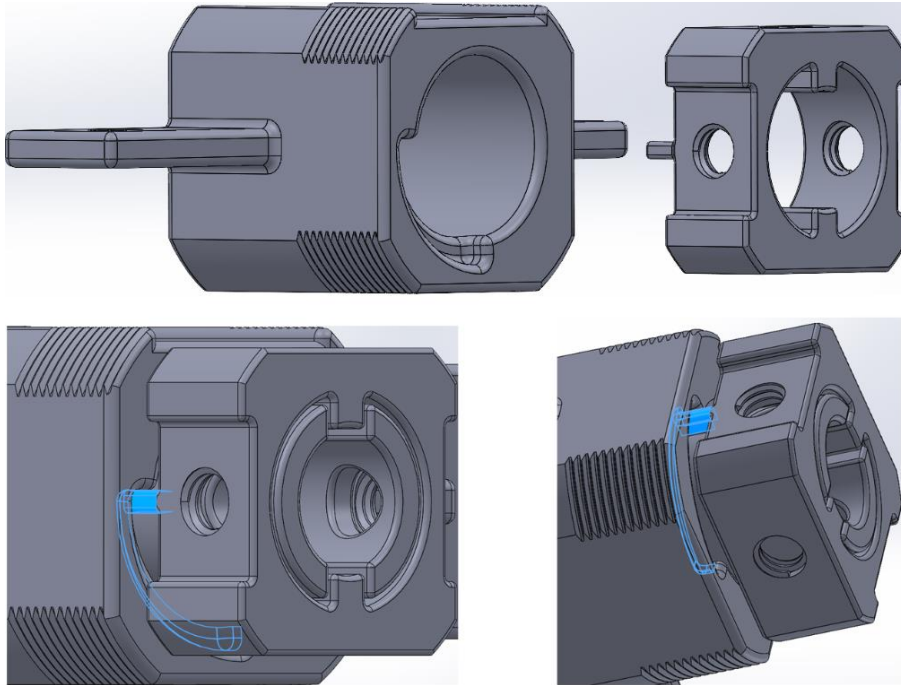
Joonis 27. Kapi uuendatud avamise mehhanismi detailid.

Hoidik ja Naagelvärk jäid põhimõtteliselt samasuguseks. Naagelvärkil eemaldati ajutine auk mutri jaoks ning muudeti veidike nii Naagelvärgi kui ka Hoidiku kuju ja suurust. Peamine muudatus on Toru kinnitumine Gerd Kanteri külge ning uus Käepideme 90° liikumisulatuse funktsioon Gerd Kanteri ja Kimalase vahel. Prototüübil oli Käepideme toru kinnitatud Gerdi külge kahe M1x10 mm poldiga, kuid need olid liiga peenikesed ning läksid hoovamehhanismi keerates lõpuks pooleks. Uuel Kanteril on kaks M3 kruvi auku ning külgede peale on jäetud süvendid, et kruvi pead mahuksid Hoidiku alla. Joonisel 28 on näha Toru, Gerd Kanterit ja Hoidikut kokku ühendatuna.



Joonis 28. Toru, Gerd Kanter ja Hoidik kokku ühendatuna.

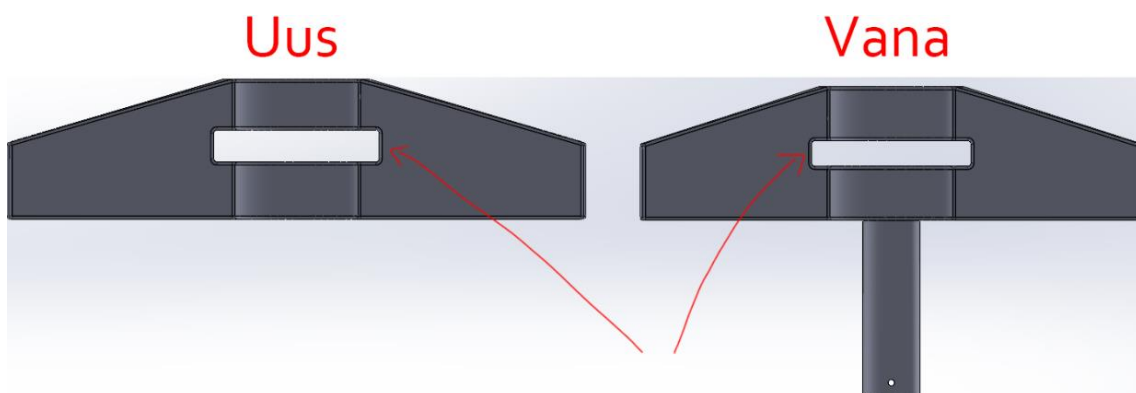
Uue Gerd Kanteri ja Kimalase vahelise 90° Käepideme liikumisulatuse funktsiooni jaoks modelleeriti Gerdile väike vars, mis libiseb Kimalases oleva kaare sees (vt Joonis 29, sinised märgistused).



Joonis 29. Käepideme 90° liikumisulatuse funktsioon Gerd Kanteri ja Kimalase vahel.

2.2.5 Kapi lukustamise mehhanismi muudatused

Analoogselt Elektroonikakarbi komponendiga, ei muudetud eriti palju kapi lukustamise mehhanismi, kuna see töötas vigadeta. Ainus muudatus oli käepidemes postkastiluku mutri augu läbimõõdu suurendamine, kuna prototüübi mudelil oli see liiga väike ning seda tuli käsitsi suuremaks lihvida.



Joonis 30. Postkastiluku mutri augu suurendus võrreldes prototüübiga.

3 Riistvara

Lahenduse realisatsioon hõlmab endas järgnevat riistvara:

- ESP32 minikit mikrokontroller,
- 12 V 1.2 A toiteplokk,
- D-sun 4R7 1136 pinge regulaator,
- S7V7F5 pinge regulaator,
- SR303 Schottky diood,
- 12 V LKmini08 solenoidlukkk,
- KY-019 JQC-3FF-S-Z releemoodul,
- KSSG1201-16 sumisti,
- 270 Ω takisti,
- Punane LED,
- PN532 v3 NFC moodul,
- 3.5-tolline Adafruit HX8357D LCD ekraan.

Võrreldes eelmise projektiga, on juurde tulnud ESP32 minikit mikrokontroller kahe ESP8266 D1 mini asemele. ESP8266 D1 mini puuduseks on ühetuumalisus ja ka GPIO väljaviikude vähesus ja eelnevalt nimetatud ESP32 mikrokontroller lahendab mõlemad probleemid. See valik muutis kogu lahenduse kompaktsemaks, mis oli ühtlasi üks töö eesmärkidest.

Kasutusele on tulnud ka solenoidi jaoks kaitsedioid SR303. Voolu kadumisel solenoidi polaarsus muutub vastupidiseks, mistõttu tekib suur tagasilöögipinge ja ilma kaitsmeta pikema aja jooksul põletaks see ära relee ja solenoidi vahelise kontakti. Kaitsedioidi olemasolul voolu kadumisel saab solenoid ennast korralikult tühjaks laadida [3]. Dioidi mudeli valik on tehtud selle põhjal, et vastupingestatud olekus see ei juhiks voolu [4]. Solenoid saab 12 V toidet ja antud dioid vajab vastupingestatult vähemalt 21 V pinget, et see hakkaks voolu juhtima [5].

Lahenduses on kasutusel ka teistsugune releemoodul. Sellele ei ole lisatud eraldi kaitsedioodi, sest antud moodulil on juba kaitsediood olemas.

Juurde on tulnud ka veel üks pingeregulaator. ESP32 mikrokontroller vajab 3.3 V pinget töötamiseks, kuid kasutusel olev releemoodul nõuab 5 V pinget, seega lõpuks oli vaja kahte toite reguleerijat.

3.1 Elektriskeem

Lahenduse riistvaralisi komponente juhib ESP32 minikit mikrokontroller. Sellega on ühendatud 3.3 V väljundiga toite reguleerija, LED koos takistiga, releemoodul, sumisti, NFC moodul ja LCD ekraan.

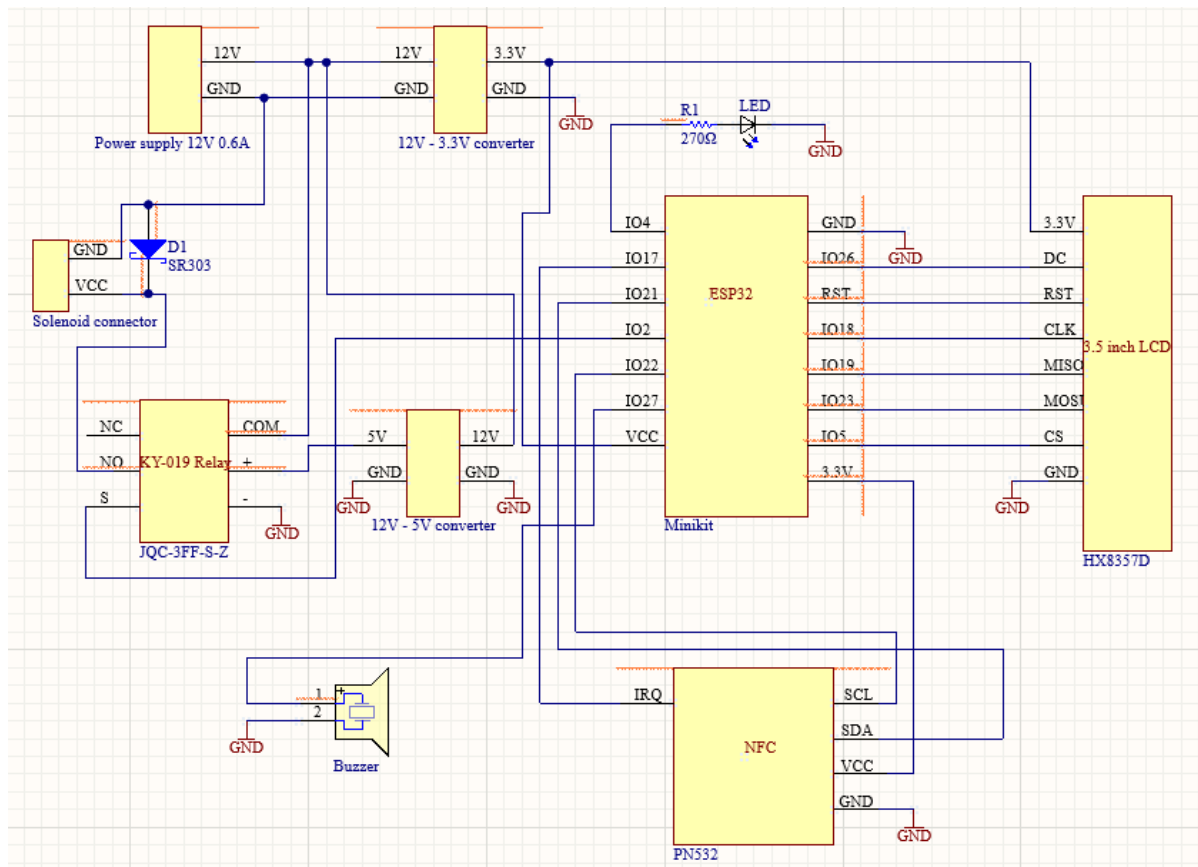
Kogu süsteemi toide on rahuldatud 12 V toiteploki abil, mille pinget muundavad ümber 5 voldiks ja 3.3 voldiks kaks pingereguleerijat. 5 V pinget kasutab releemoodul, mis omakorda lülitab 3.3-voldise loogikasisignaali abil [6] toiteploki tuleva 12 V pingega solenoidlukku. Relee puhul on kasutusel NO väljaviik, sest solenoid peab normaalingimustel olema avatud.

Teise toite reguleerija väljundpinget kasutab mikrokontroller ja ekraan. Joonisel 31 on näha, et NFC moodul kasutab enda toiteks mikrokontrollerist tulevat sama suurt pinget, kuid tegelikult on ka võimalik seda toita sama toite reguleerija 3.3-voldise väljundpingega. Visuaalse arusaadavuse huvides on see nii jäetud. NFC mooduliga suhtlemiseks on kasutatud I2C protokoll ja kõik vajalikud ühendused on tehtud vastavalt dokumentatsioonile [7]. Kasutusel on ka mooduli IRQ väljaviik, et kontroller ei peaks pidevalt NFC moodulilt küsima, kas kaart on tuvastatud. Sedasi saadab kaardilugeja lihtsalt teatesignaali juhtseadmele, kui kaart on skaneeritud.

Skeemis on ka perifeerse eesmärgiga sumisti ja punane LED. LEDile on lisatud üks 270 Ω takisti, et LED jääks pingega enda tööpiirkonda.

Algselt oli lahenduses kasutusel üks 3.2-tolline Raspberry Pi jaoks mõeldud LCD ekraan. Draivereid, et see ESP32 peal tööle saada, aga ei olnud nii, et viimasel hetkel vahetati ekraan 3.5-tollise Adafruit HX8357D LCD ekraani vastu välja. Viimast ekraani aga ei jõutud katsetada, kuid skeem peaks teoreetiliselt töövõimeline olema.

Kommunikatsioon mikrokontrolleri ja ekraani vahel toimub läbi SPI protokoll. Puutetundlikkuse tagamiseks on kasutatud resistiivse puuteekraani funktsionaalsust. Sedasi on vajaminevate ekraani ja mikrokontrolleri vaheliste ühenduste hulk võimalikult väike tänu millele on moduli paigaldus natuke lihtsam ja tagatud on ka pisut suurem töökindlus. Täpsed väljaviikude ühendused on tehtud vastavalt esemete dokumentatsioonidele [8] [9].



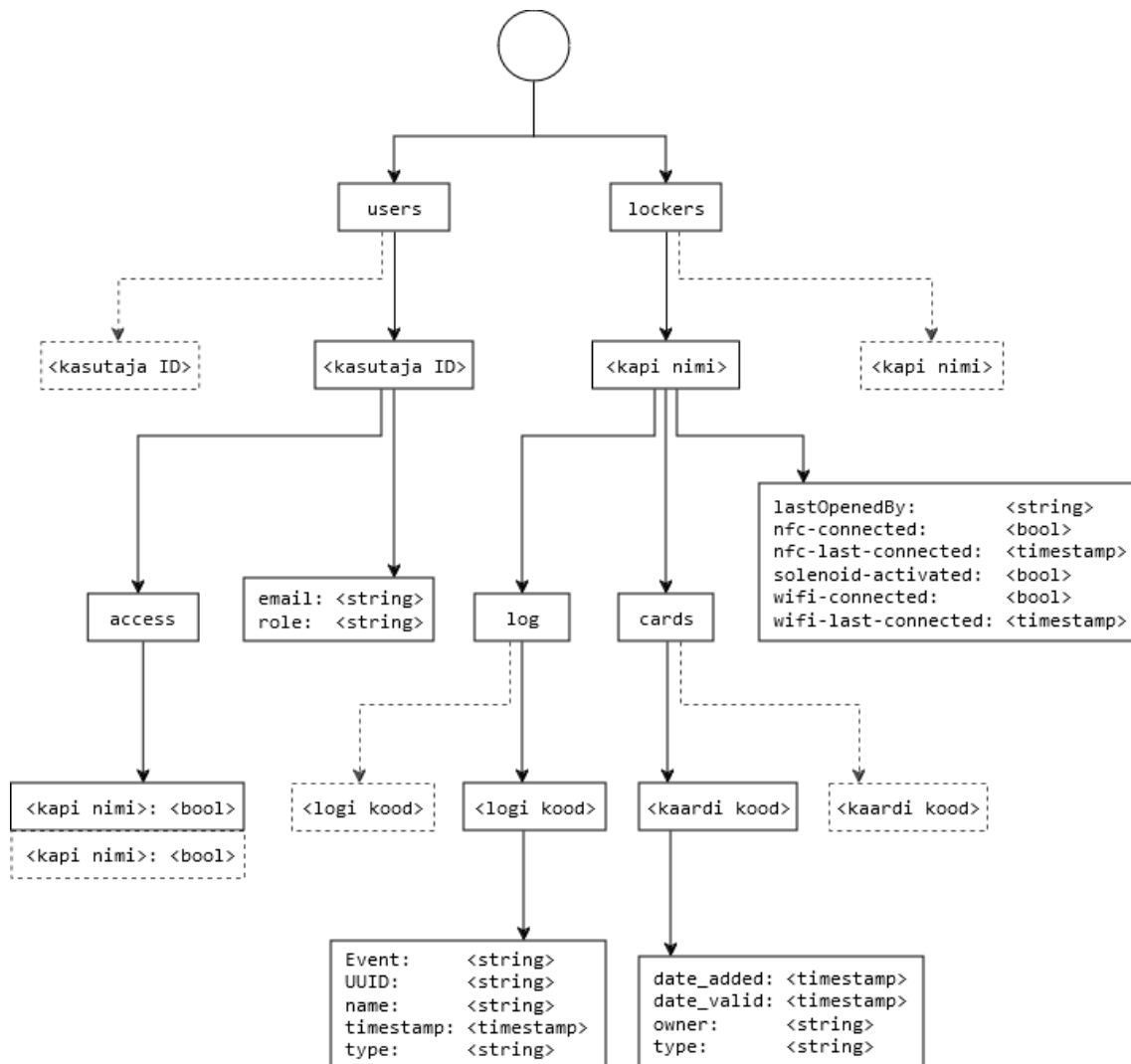
Joonis 31. Laborikapi elektriskeem.

4 Tarkvara

Edasiarenduse tarkvara jaguneb kaheks suuremaks osaks - mikrokontrolleri püsivaraks ja veebiliidese tarkvaraks. Varasemas projektis hargnes püsivara veel omakorda kahe mikrokontrolleri vahel. Ekraani juhtiva selli ja lukustussüsteemi juhtiva meistri asemel võeti edasiarenduse raames kasutusele üks kahetuumaline ESP32, mis teeb paralleelselt mõlema varasemalt kasutatud ESP8266 töö ära. See mitte ainult ei lihtsusta skeemi, vaid on ka energiasäästlikum ja kiirem [10]. Kapi ja kasutajaliidese vaheliseks suhtluseks kasutatakse ühist Google Firebase'i reaalaja andmebaasi [11]. Tarkvara arendamise käigus kasutati vana projekti kiiremaks ja paremaks mõistmiseks suure keelemudeli ChatGPT abi [12].

4.1 Andmemudel

Joonisel 32 on näha üldine graafiline ülesehitus JSON andmemudelist. Andmemudelis on enamasti säilinud samad väljad, mis varasemas projektis. Kuna mooduli universaalseks muutmise käigus loobuti magnetsensori kasutamisest, on vastavad väljad andmemudelist eemaldatud. Mitme kapi toe lisamiseks tuli mudelile teha mõningaid muudatusi olemitevahelistes suhetes, muutes puu sügavamaks. Lisandunud on uus haru *users*, et lisada tugi veebirakenduse ligipääsu haldamiseks. Andmepuu juurest väljuvad tipud *users* ja *lockers*, mis eraldavad kahte rakenduse põhiosa: veebi- ja kapikasutajaid.



Joonis 32. JSON formaadis andmemudel.

4.1.1 Veebikasutajate haru *users*

Users väljale saab alluda lõpmatu arv (kuni andmebaas lubab) Firebase'i kasutaja ID-sid. Unikaalne ID luuakse igale kasutajale Firebase'i autentimissüsteemi poolt automaatselt. Vastava kasutaja lisamine andmebaasi on aga veebirakenduse vastutada (vt peatükk 4.3). Igast kasutaja ID-st väljuvad tipud *access* ning kasutaja e-maili ja rolli hõlmavad nimi-väärtus paarid. *Access* tipp on ülemaks lõpmatule hulgale nimi-väärtus paaridele, mis hoiavad endas vastava kapi nime ja tõeväärtust, määramaks selle, kas kasutajal on ligipääs vastavale kapile või mitte. Kapinimelise tipu puudumine on võrdväärne ligipääsukeeluga.

4.1.2 Kapikasutajate haru *lockers*

Juurest väljuva *lockers* alluvateks on lõpmatu hulk tippe kappide nimedega. Nende tippude nimed peavad olema vastavuses *users* haru *access* tipu kirjetega. Rakenduse töökohalt on oluline, et igal kapil oleks ligipääs ka iseendale (vt peatükk 4.2). Igast kapinimega tipust väljub kolm tippu – kapi avamisajalugu jälgiv *log*, kapile ligipääsevate ISIC kaartide kogu *cards* ja kapi seisu haldavate omaduste nimi-väärtus paaride kogumik. Viimane koosneb järgnevatest omadustest: *lastOpenedBy*, *nfc-connected*, *nfc-last-connected*, *solenoid-activated*, *wifi-connected* ja *wifi-last-connected*. *LastOpenedBy* omab väärtusena kaardi omaniku nime, kes viimasena kapi avas. *Solenoid-activated* on tõeväärtuslik omadus, mis vastab kapi lukuolekule – kui väärtus on tõene, on solenoid aktiivne ning solenoidluku keel on tõmmatud avatud asendisse. Ülejäänud neli nimi-väärtus paari vastavad oma ingliskeelse nime järgi, kas Wi-Fi või NFC mooduli ühendusoleku tõeväärtusele või viimasele ühendushetkele ajatempli näol.

Tipp *cards* saab omada lõpmata palju unikaalseid kuueteistkümnendsüsteemis olevaid NFC kaartide koode. Igal kaardikoodiga tipul on alamtipp nelja nimi-väärtus paariga. *Date-added* ja *date-valid* on ajatemplid, mis on vastavalt võrdsed kaardi lisamis- ja aegumishetkele. Väli *owner* hoiab endas kaardi omaniku nime. Viimasena on määratud ka *type*, mis saab olla kas „*teacher*“ või „*student*“. Kui kaardil on üks kahest rollist ning ajatempel ei viita aegumisele, saab ta kappi avada. „*Teacher*“ rolliga kaart saab peale avamise lisada mooduli ekraanilt ka kasutajaid juurde.

Tipust *log* saab väljuda lõpmata palju unikaalse koodiga alamtippe, igaüks nendest vastab ühele logisissekandele. Iga logi viitab lehele, kus on viis nimi-väärtus paari. *Event* hoiab kirjeldust juhtumist – rakendus lisab vaid kirjeldusi *Attempted Open* ja *Opened*, mis vastavad avamiskatsele ja õnnestunud avamisele. UUID väli vastab kaardi koodile, mis vastava juhtumi põhjustas. Selle väärtus võib olla ka „-“, juhul kui kaardi lugemine ebaõnnestus või sissekandega on seotud hoopis veebikasutaja. *Name* väli vastab veebikasutaja või kaardi omaniku nimele, kui see on teada. Vastasel juhul omandab väli väärtuse „TBD“. *Type* on võrdne kasutaja rolliga, tundmatu puhul on väärtuseks „*unknown*“. Lisaks on nimi-väärtus paarina veel ajatempel nimega *timestamp*, tähistamaks juhtumi ajahetke.

4.2 Autentimine ja autoriseerimine

Projekti autentimislahendusena kasutatakse Firebase'i poolt pakutavat Firebase Authentication teenust. Täpsema seadistusena on määratud autentimispakkujana „Email/Password“, mille abil saab luua emailipõhiseid parooliga turvatud kasutajaid. Paroolide turvalise salvestamise ja haldamisega tegeleb Firebase ise automaatselt. Kuigi kasutajad on meilipõhised, ignoreerib rakendus e-meiliga kasutaja kinnitamist ja paroolitaastamise funktsiooni, sest kasutajaid peab saama lisada vaid administraator, mitte potentsiaalne kasutaja ise. See takistab väärkasutajate loomist ning võimalike turvaaukude ärakasutamist. Kuna email ei lisa kasutajale funktsionaalsust, on veebiliideses see peidetud ja kuvatud on selle asemel kasutajanimi, mis tegelikkuses autentimisel ja autoriseerimisel muudetakse emailiks domeenilõpuga „@nutikapp.ee“.

Mitme kapi tugi tekitas vajaduse rollipõhise autoriseerimise järgi, et iga kasutaja ei saaks iga kapi seadeid muuta ja kasutajaid lisada. Selleks lisati rollid „admin“, „user ja „locker“ – esimene saab muuta viimaste ligipääsu vastavale kapile. Autoriseerimine on lahendatud Firebase'i reaalaja andmebaasi reeglitega (vt lisa 2). Need reeglid kehtivad kõigele ja kõigile, kes üritavad andmebaasile ligi pääseda.

Reeglistik toimib põhimõttel, et kui kellelgi on ülemtipule ligipääs, on tal ligipääs ka kõikidele tema alamtipudele. Eraldi ligipääsureeglid on kehtestatud kogu *lockers* objektile, kindla nimega kapile, kogu *users* objektile ja kindla nimega kasutajale. Kõik reeglid on piiratud tingimusega, et autentimine ei tohi puududa, vältimaks väljalogituna ligipääsemist.

Oli vajalik tagada, et administraatorid saaksid ligi kõikidele kappidele, kasutajad aga ainult nendele, millele neil on luba antud. Selleks on kirjutamis- ja lugemisõigused kogu *lockers* objektile on antud vaid juhul, kui sisselogitud kasutaja roll andmebaasi *users* harus on „admin“. Individuaalse kapi lugemis- ja kirjutamisõigused on aga ka antud neile, kellel on *access* all alamtipuna nimi-väärtus paarina vastava kapi nimi ja tõeväärtus *true*.

Oluline oli ka piirata teiste kasutajate ligipääsu kõigi andmetele. Administraatorid aga vajavad kasutajate loetelule täieliku ligipääsu, et veebikasutajaid hallata. See realiseeriti sarnaselt eelnevale - kui *users* harus on sisselogitud kasutajal „admin“, on tal lugemis- ja kirjutamisõigus kogu *users* harus. „User“ või „locker“ tüüpi kasutaja saab vastava

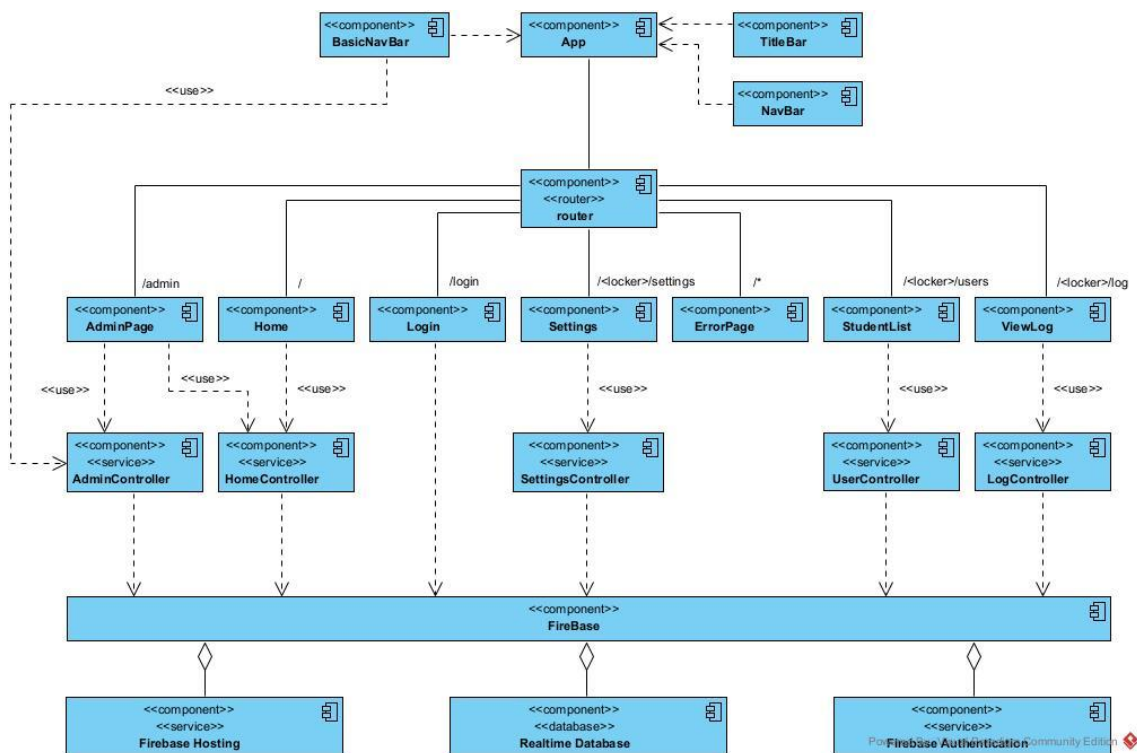
kasutaja andmeid lugeda vaid siis, kui päritud kasutaja ID on sama, mis sisselogitud kasutaja ID.

Selline autoriseerimisskeem tekitas vajaduse lisada kasutajad ka kappidele. Varasemas projektis pidi kapiseade samuti sisse logima, et andmebaasile ligi pääseda, kuid tegi seda läbi arendusadministraatori kasutaja. Vältimaks seda, et kapi kaudu saaks kuidagi ligi kogu andmebaasile, lisati kappide jaoks eraldi roll „*lockers*“ ja igale kapile oma kasutaja. Eraldi roll on hea kapile ja tavakasutajatele mõeldud kontode eristamiseks, kuid käitub sellele lisaks ka veebirakenduses teisti kui roll „*user*“, kui teda lisada või muuta (vt peatükk 4.3.5).

4.3 Veebirakendus

Veebirakendus on endiselt üles ehitatud raamistikuga Vue.js, mis ehitab üles SPA tüüpi veebilehe. Vue.js laiendab HTML-i märksõnadega, et elemente saaks TypeScripti kasutades dünaamiliselt muuta. SPA-le omaselt, kasutab Vue.js *router*'it, et lehe uuestilaadimise asemel kuvada URL-ile vastava *View* ehk vaate sisu dünaamiliselt *App* komponendile [13].

Antud edasiarenduse raames tuli täiendada kõiki komponente peale *ErrorPage*'i. Vanad *view*'d vajasisid vaid mõne argumendi muutust ja rollikontrolle, neid juhtivad komponendid aga suuremaid muutusi. Lisati uus *view* „*AdminPage.vue*“, navigatsiooniriba „*BasicNavBar.vue*“ ja juhtkomponendid „*AdminController.ts*“ ning „*HomeController.ts*“. Kõiki lisandunud komponente ja nende suhteid ülejäänud rakendusega on näha komponentdiagrammilt joonisel 33. Järgnevates peatükkides on lahti seletatud vaid uued komponendid ja tehtud muudatused, varemloodu täpsema kirjelduse leiab eelneva projekti aruandest [1].



Joonis 33. Veebirakenduse komponentdiagramm.

4.3.1 Komponendid *router.ts* ja *app.vue*

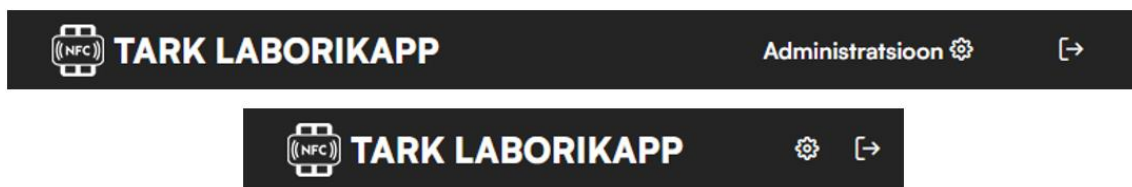
Et võimaldada mitme kapi tugi, pidi „*Router.ts*“ muutuma dünaamiliseks. Kindla kapi seadetele, kasutajatele ja logidele ligipääsemiseks pidi looma igale kapile oma unikaalse URL-i. Selleks on pandud *router* võtma vastu kõiki „/settings“, „/logs“ ja „/users“ linke, enne mida on veel suvalise nimega marsruut. See marsruut muudetakse parameetriks „*lockerName*“. Kui kasutaja üritab ligi pääseda seadetele kapil nimega „kapp1“, peab ta minema lehele „/kapp1/settings“. Lehele minnes annab *router* vastava parameetri nime „kapp1“ *propina* kaasa komponendile, et ta teaks andmebaasist vastavat kappi otsida. Peale selle on lisatud uus *route AdminPage*’ile aadressil „/admin“. Seadete, logide ja kasutajate lehekülgede nimepäisesse on samuti lisatud vastava kapi nimi.

Rakenduse tuumleheküljele „*App.vue*“ lisandus uus navigatsiooniriba „*BasicNavBar.vue*“ ja uuendatud tingimused, millist navigatsiooniriba peab rakendus hetkel kuvama. Lehtedel, mis on seotud individuaalsete kappidega ehk „*Settings.vue*“, „*StudentList.vue*“ ja „*ViewLog.vue*“ kuvavad endiselt „*NavBar.vue*“ komponenti, mis võimaldab liikuda eelnevalt nimetatud lehtede vahel. „*BasicNavBar.vue*“ komponenti, kus seda funktsionaalsust pole, kasutavad veebilehed aga juhul, mille URL on „/“, „/admin“ või

lehekülje nimi „*NotFound*“. Marsruudil „*/login*“ kasutatakse viideteta riba „*TitleBar.vue*“, sest enne sisselogimist ei saa nagunii teistele lehtedele ligi.

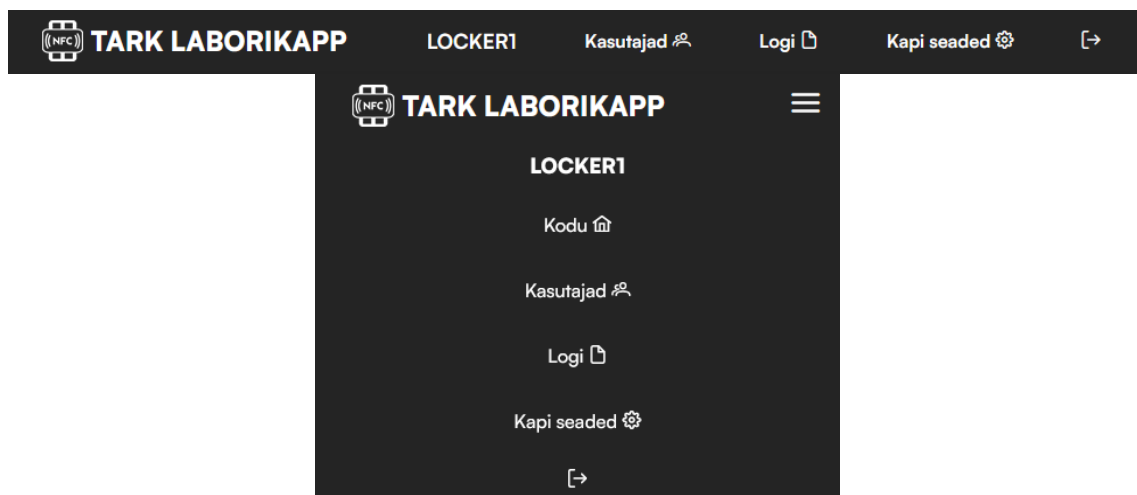
4.3.2 Navigatsiooniribade komponendid

Mitme kapi toe lisamisel tekkis vajadus uue navigatsiooniriba järele, sest „*NavBar.vue*“ ei saanud olla enam kõikidel vaadetel. Pea-, administratiiv- ja „*Not Found*“ lehed on üleüldised, ega pole seotud ühegi kapiga, seega ei saa nendele lisada viiteid kappide seadete-, logide- ja kasutajahaldamislehtedele, sest nad ei tea millisele kapile viidata. Selleks loodi uus navigatsiooniriba „*BasicNavBar.vue*“, mis on sisuliselt ilma kapinavigatsioonita „*NavBar.vue*“. Samuti kasutab uus riba komponendi „*AdminController*“ meetodit „*getUserRole*“, et kuvada viit administratiivlehele juhul kui kasutaja on administraator (vt joonist 34).



Joonis 34. „*BasicNavBar.vue*“ administraatori vaates erinevate aknasuuruste puhul

Lisaks muudeti ka vana navigatsiooniriba „*NavBar.vue*“, kust eemaldati arvutivaates viide „Kodu“ ja lisati veidi ajakohasem kasutajakogemus klikitava logo näol. Nii telefoni- kui ka arvutivaatesse lisati viide kapi nimele, et kasutaja teaks, millise kapi seadeid, logi või kasutajaid ta hetkel näeb (vt joonis 35).

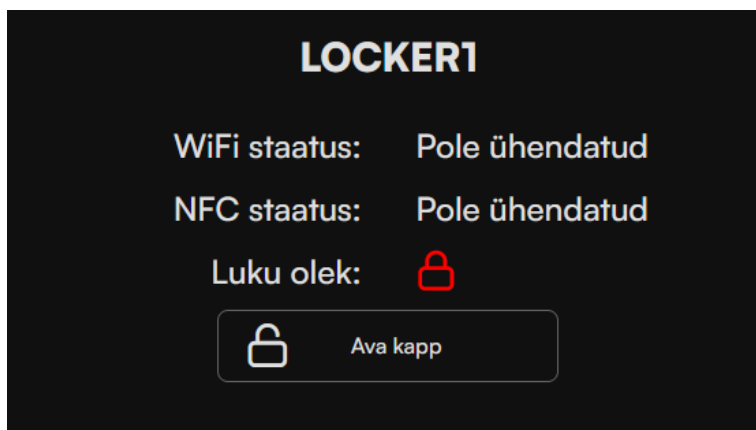


Joonis 35. „*NavBar.vue*“ erinevate aknasuuruste puhul.

4.3.3 Kapi seadete-, logide- ja kasutajahaldamisvaadete komponendid

Kapi seadeid, logisid ja kasutajaid juhtivates komponentides muutus andmete hankimine. Iga meetod komponentides „*UserController.ts*“, „*LogController.ts*“ ja „*SettingsController.ts*“, mis lisab või otsib andmebaasist midagi, on saanud juurde parameetri „*lockerName*“. Mitme kapi toe tõttu vajavad nende komponentide meetodid andmebaasist vastava kapi andmete otsimiseks kapi nime. *Routeriga* kaasa antud *prop* „*lockerName*“ jõuab kõigepealt *view*'sse ja sealt seejärel antakse ta argumendina edasi ühele nende komponendi meetoditest.

Peale *propi* saatmise vastavale meetodile, on lisatud vaatedetesse „*StudentList.vue*“, „*ViewLog.vue*“ ja „*Settings.vue*“ ka veateated, juhul kui kasutajal pole õigusi kapile ligi pääsemiseks. Oluline muutus tehti vaatele „*StudentList.vue*“, kus muudeti kasutaja kaardi sisestusmalli nõnda, et sinna jääksid tühikute asemel sidekriipsud. Firebase ei luba olemi nimesse tühikuid panna, see oli ka põhjuseks miks vana kapiseade töötamast lakkas (vt peatükk 4.4.3). Samuti on lisatud pisikesi mugavus- ja kujundusmuudatusi, näiteks seadete lehekülje paneelil kuvatakse nüüd ka kapi nimi, eemaldati ebavajalik väli „ukse olek“, parandati suurusemuutustele reageerimine ja viga, kus nupp „Ava kapp“ muutus vajutades piklikuks (vt Joonis 36).



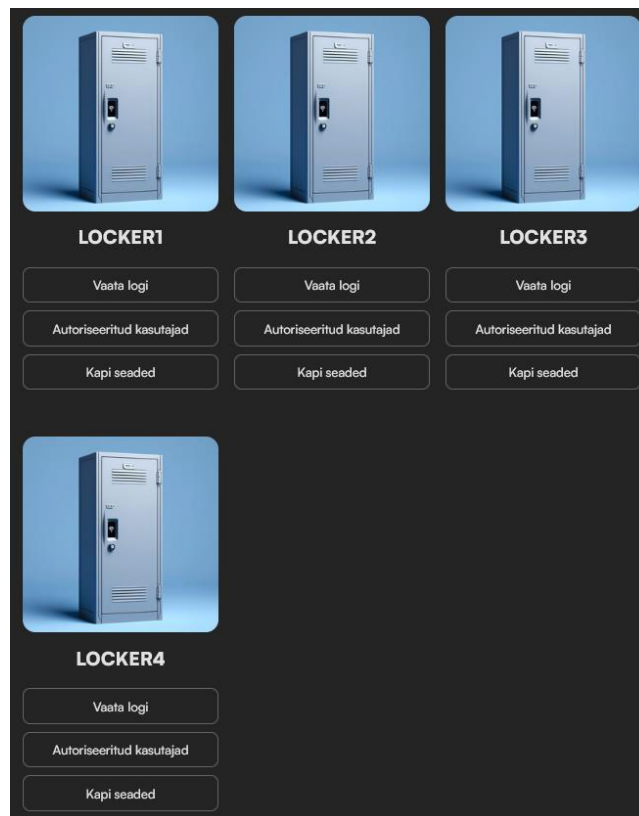
Joonis 36. *Settings.vue* kapiseadetepaneel.

4.3.4 Pea- ehk koduvaate komponendid

Juurele viitav URL, ehk lehekülje koduvaade „*Home.vue*“, kuvab kõik kapid ja nende seadetele, kasutajatele ning logidele suunavad nupud, millele kasutajal ligipääs on. Varasemalt oli see püsikodeeritud, sest rakenduses oli vaid üks kapp, millele link suunama pidi. Pealehe dünaamiliseks muutmiseks oli vaja luua juhtkomponent „*HomeController.ts*“, mis päriks kasutaja ligipääsuandmeid.

Meetod „*getUserLockers*“ võtab parameetrina sisse kasutaja-ID, et leida andmebaasist vasted kindla *users* haru *access* objektist. Iga tõese nimi-väärtus paari korral lisatakse vastava kapi nimi massiivi ja tagastatakse meetodi kutsujale. Vasteid mitte leides tagastatakse tühi massiiv.

Meetod „*getAllLockers*“ on mõeldud vaid administraatorile, sest pärib kogu *lockers* haru alamliikmed, millele on ligipääs seadistatud reeglite kohaselt vaid kasutajatel rolliga „*admin*“ (vt peatükk 5.2). Meetod otsib andmebaasi *lockers* harust üles kõikide kappide nimed ja tagastab need massiivina. Vaade „*Home.vue*“ kasutab ka komponenti „*AdminController.vue*“, et teada saada kasutaja roll ja valida seejärel üks „*HomeController.vue*“ meetoditest ligipääsuga kappide hankimiseks. Peale seda lisatakse iteratiivselt kõik kapid ja nende lingid HTML elementidena leheküljele. Korrektse kuvamise jaoks on kasutatud CSS *grid display* tüüpi, mis reageerib ka õigesti vastavalt internetilehitseja aknasuurusele (vt joonis 37).



Joonis 37. „*Home.vue*“ vaade nelja kapiga.

4.3.5 Administratiivvaate komponendid

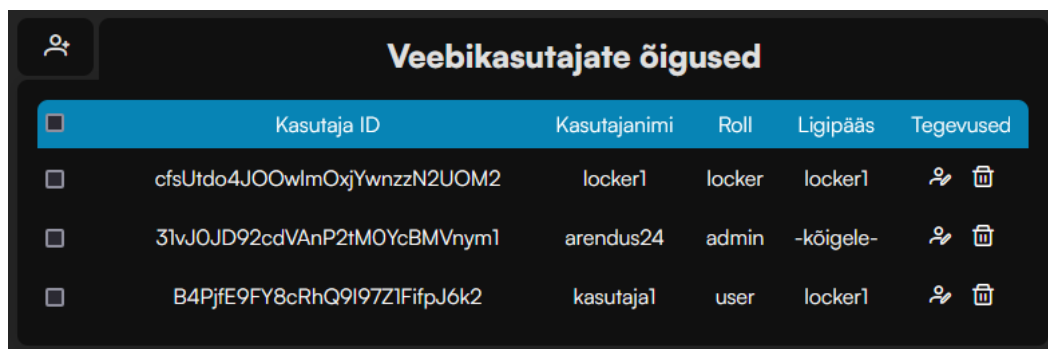
Mitme kapi tugi tekitas vajaduse kasutajate administreerimise järele. Kuigi kasutajaid saaks hallata ka lihtsalt andmebaasi kaudu, on palju mugavam teha seda läbi

kasutajaliidese. Selleks lisati rakendusse vaade „*AdminPage.vue*“ ja komponent „*AdminController.ts*“.

Komponent „*AdminController.ts*“ on kaks meetodit kasutajaandmete hankimiseks. „*getUserRole*“ hangib sisse antud kasutaja-ID argumendi alusel vastava kasutaja rolli andmebaasi *users* harus. Tavakasutaja saab andmebaasireeglite tõttu hankida vaid enda rolli, administraator aga kõigi. See meetod on antud komponendis ainuke meetod, mida saab kasutada ka tavakasutaja. „*GetAllUsersAndSort*“ hangib kõik kasutajad *users* harus, ning võtab lisaks sisse ka parameetrid „*sortField*“ ja „*sortOrder*“, et tagastatud tabel oleks soovitud järjestuses. Sorteerimiseks kasutab ta eraldi komponendi meetodit „*sortTable*“.

„*AdminController.ts*“ komponendis on veel kolm meetodit kasutajate haldamiseks: „*addUser*“, „*updateUser*“ ja „*removeUser*“. Esimene võtab parameetritena sisse kasutajanime, parooli, rolli ja ligipääsu kirje, loob kõigepealt Firebase Authenticationi kaudu uue kasutaja ning seejärel lisab kasutajaandmed ka reaalaja andmebaasi. Meetod tagastab loodud kasutaja andmed, koos loodud kasutaja ID-ga. Kasutajanimele liidetakse otsa meilidomeen „@nutikapp.ee“. Uue kasutaja loomiseks Firebase’is tuli kasutada meetodit „*createUserWithEmailAndPassword*“, kuid antud meetod logib peale uue kasutaja loomist kasutaja looja ka sinna kasutajasse sisse. Vältimaks, et administraator lüüakse peale kasutaja loomist sessioonist välja, mistõttu ei jõua uuele kasutajale vastavad kirjed andmebaasi *users* harusse, loodi taustale uus ajutine sessioon, mis loob kasutaja administraatori eest ja logib kohe ka välja [14]. „*UpdateUser*“ meetodi kaudu saab andmebaasis olevaid kasutajaandmeid muuta. „*RemoveUser*“ kustutab vastava kasutaja andmebaasist. Küll aga ei eemalda viimane meetod kasutajat Firebase’ist. Turvalisuse eesmärgil saab rakenduse kaudu oma Firebase’i konto kustutada ainult kasutaja ise. Seetõttu peab kontode lõpliku kustutamise tegema Firebase’i keskkonnas. Nii „*addUser*“ kui ka „*updateUser*“ käituvad vastavalt argumendina saadud rollile erinevalt. Kui kasutaja rolliks on „*locker*“, määratakse talle ligipääsetavaks kapiks automaatselt iseennast ja lisatakse vastavad kirjed uuest kapist ka andmebaasi *lockers* harusse. Samamoodi eemaldavad „*updateUser*“ ja „*removeUser*“ ebavajalikud andmebaasi kirjed.

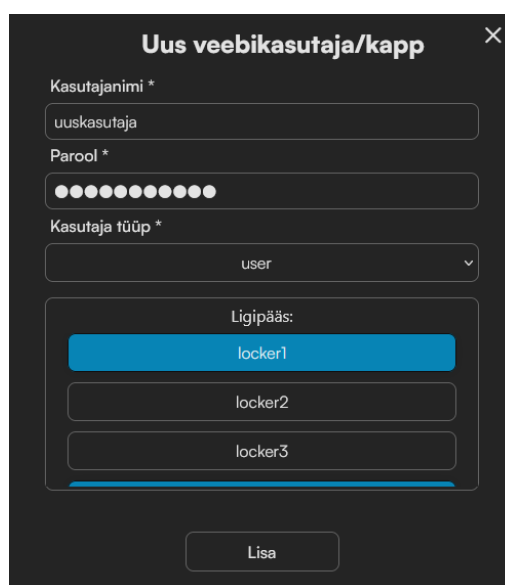
„AdminPage.vue“ kuvab olemasolevate kasutajate ID-d, kasutajanimed, rollid ja ligipääsuandmed tabelis, sarnaselt vaatele „StudentList.vue“ (vt joonis 38). Vaates on võimalik sorteerida andmeid vastavalt kasutaja ID-le, kasutajanimele ja rollile, klikkides vastavale kategooriale. Vasakult ülevalt on võimalik uusi kasutajaid lisada. Paremtalt saab vastava kasutaja andmeid muuta, või üldse kustutada. Samuti on võimalik teha kasutajatest mitmikvalik, et neid korraga kustutada.



☐	Kasutaja ID	Kasutajanimi	Roll	Ligipääs	Tegevused
☐	cfsUtdo4JOOwImOxjYwnzzN2UOM2	locker1	locker	locker1	
☐	31vJOJD92cdVAnP2tMOYcBMVnym1	arendus24	admin	-kõigele-	
☐	B4PjfE9FY8cRhQ9l97ZlFifpJ6k2	kasutaja1	user	locker1	

Joonis 38. AdminPage.vue tabelivaade.

Joonisel 39 on kujutatud kasutaja lisamise vaade. Kasutaja lisamis- ja muutmisaken on sarnased, viimases on peidetud vaid parool ja lukustatud kasutajanime lahter. Kasutaja rolli muutmine toimib valikuribaga. Vastavalt rollile kuvatakse ka kõige alumine lahter „Ligipääs“, kust on võimalik hallata kasutaja kappide õiguseid. Kappide valik tekib vaid „user“ rollil, „locker“ rollil määratakse automaatselt ligipääs iseendale ja „admin“ eraldi ligipääsu ei saagi, sest antud rollil on ligipääs igalepoole automaatselt.



Uus veebikasutaja/kapp

Kasutajanimi *

uuskasutaja

Parool *

●●●●●●●●

Kasutaja tüüp *

user

Ligipääs:

locker1

locker2

locker3

Lisa

Joonis 39. AdminPage.vue veebikasutaja lisamise vaade.

4.4 Püsivara

Püsivara pidi seadistama ümber uuele platvormile ESP32, samuti pidi arvestama teiste riistvaraliste muutustega, nagu puuteekraan, seadistatud GPIO väljaviigud ja *flash* mälu suurus. Oluline samm oli tuvastada ja parandada viga, mis põhjustas vana seadme vigase töötamise. Püsivaras pidi samuti arvestama mitme kapi toe olemasoluga ja viima sisse vastavad muudatused, sest ka mikrokontroller vajab ligipääsu andmebaasile.

4.4.1 Projekti seadistus

Eelnevad autorid võtsid kasutusele Visual Studio Code'i laienduse PlatformIO, sest see muudab projekti ülesseadmise mugavamaks [1]. Kuigi valik oli hea, oli ülesseadmine siiski tülikas, sest nende keskkonna seadetes oli puudusi. Selle asemel, et lisada kasutatud teegid PlatformIO seadistusse, lisasid nad kasutatud teegid otse koodihoidlasse. See põhjustab aga olukorra, et teegid ei uuene automaatselt ja nendega ei lisandu teegi toimimiseks vajalikke kompilaatoriargumente.

Kõik välised teegid eemaldati koodihoidlast ja lisati siis tagasi, kasutades PlatformIO seadistusfaili „platformio.ini“. Samuti määrati uueks platvormiks MH-ET LIVE ESP32, et teist mikrokontrollerit kasutada. PlatformIO seadistusse märgiti ka ekraani seaded, ekraanikontrolleri tüüp ja mikrokontrolleri väljaviigud, vastavalt välisele teegile „TFT_eSPI“ ja Adafruiti dokumentatsioonile [15] [16]. Teekide täpsemast kasutusest projektis saab lugeda vana projekti aruandest [1].

Kapi esmase ülesseadmise lihtsustamiseks tulevikus lisati failid „*PinConfig.h*“ ja „*FirestoreConfig.example.h*“. Esimeses on võimalik muuta globaalselt väljaviike vastavalt vajadusele. Teine aga teeb palju lihtsamaks eelnevate autorite lahenduse kopeerida vajalik kood Firestore'i seadistamiseks väljaspoolt – fail tuleb lihtsalt duplikeerida, nimeks panna „*FirestoreConfig.h*“, ning asendada failis olevad lüngad oma seadistatava kapi nime, Wi-Fi SSID, parooli, Firestorei API võtme, ühendusstringi ja kasutajakonto andmetega.

4.4.2 Platvormivahetusest tingitud muudatused

Platformivahetusest ja arenduskeskkonna muutumisest tekkis projektis hulk vigu, mida oli vaja parandada. C++ kompilaatori muutus tekitas vajaduse „FBServer.cpp“ failis muuta funktsioonide käitumist – objektid pidi looma enne funktsiooni kasutamist ja

nende aadressid andma sisse argumendina, sest funktsiooni kaudu väärtuse tagastamine ei toiminud. Platvormimuutuse tõttu asendati viited Arduino „ESP8266WiFi.h“ teegile, mille asemel võeti kasutusele sisseehitatud „WiFi.h“. Programmist eemaldati ka kõik, mis oli mõeldud magnetsensorile, sest seda ei kasutatud.

4.4.3 Vana kapiprojekti vigade parandus

Püsivaras oli kõige olulisem üles leida viga, mis põhjustas vana kapiprojekti töötamise lakkamise. NFC mooduliga kaardi skännimisel osutus, et Firebase'i päringud ei lähe läbi ja tagastatakse „Access denied“. Koodi uurimisel ja katsetamisel selgus, et päringud lähevad läbi vaid siis kui päritavas koodis pole tühikuid. Nõnda muudetigi andmebaasis, püsivaras ja veebirakenduses ümber kõik kohad, kus kasutati kaardikoodi. Tühikute asemel võeti kasutusele sidekriipsud, et säilitada loetavus. „Nfc.cpp“ olevas funktsioonis „convertToString“ liideti iga kahe kuueteistkümnendkujulise järel „-“, sellega oligi probleem lahendatud.

4.4.4 Mitmetuumaline lahendus

Kõige suurem muutus, mis oli vaja püsivarasse sisse viia, oli kahe mikrokontrolleri koodi viimine ühe kontrolleri kahe tuuma peale. Selleks kasutati FreeRTOS reaalaaja operatsioonisüsteemi, mis haldab kahe tuuma käitumist automaatselt [17].

Ust juhtiv programmiosa otsustati panna tuumale 1, ekraani juhtiv osa tuumale 0. Programmi alusfaili „main.cpp“ tuli lisada *TaskHandle_t* tüüpi muutuja, millele „setup()“ osas määratakse funktsiooniga „xTaskCreatePinnedToCore“ ülesanne, mida antud muutuja jälgima peab. Antud funktsioonis määratakse ära ka ülesande prioriteet, pinu suurus ja tuum, millel ta täidetakse. Kuna varasema töö koodid olid hästi eraldatud, sai suure vaevata luua igaveses tsüklis oleva funktsiooni „DoorController“, mis hõlmas endas varasema projekti „master“ haru „main“ osa. Tavalisse loop funktsiooni lisati aga „display“ haru „main“ kood, mis automaatselt jookseb tuumal 0. Kuna vana projekti ekraanikood oli kõik ühes failis, loodi failid „lockerDisplay.cpp“ ja „lockerDisplay.h“, kuhu tõsteti kõik ekraanikoodi meetodid, et jätta „main.cpp“ võimalikult selgeks.

Tuumadevaheliseks suhtluseks kasutati FreeRTOSi tüüpi *QueueHandle_t* ning meetodeid „xQueueSend“ ja „xQueueReceive“. Igas kohas programmis, kus toimus „HardwareSerialiga“ jadaandmeside läbi RX ja TX viikude, asendati see suhtlusviis ühega kahest meetodist vastavalt sellele, kas andmeid saadetakse või kuulatakse. Kui üks

tuum tahab teisega suhelda, peab ta vastava sõnumi järjekorda panema. Kuna sõnumi kuulamised on tsüklilised ja pidevad, siis ei ole vaja *interrupti* kasutada, sõnum saadakse teisel pool nagooni peagi kätte. Põhimõttelt jäi suhtlus samasuguseks – ainuke erinevus on see, et nüüd suhtlevad omavahel tuumad, mitte mikrokontrollerid.

5 Mooduli paigaldamise juhend

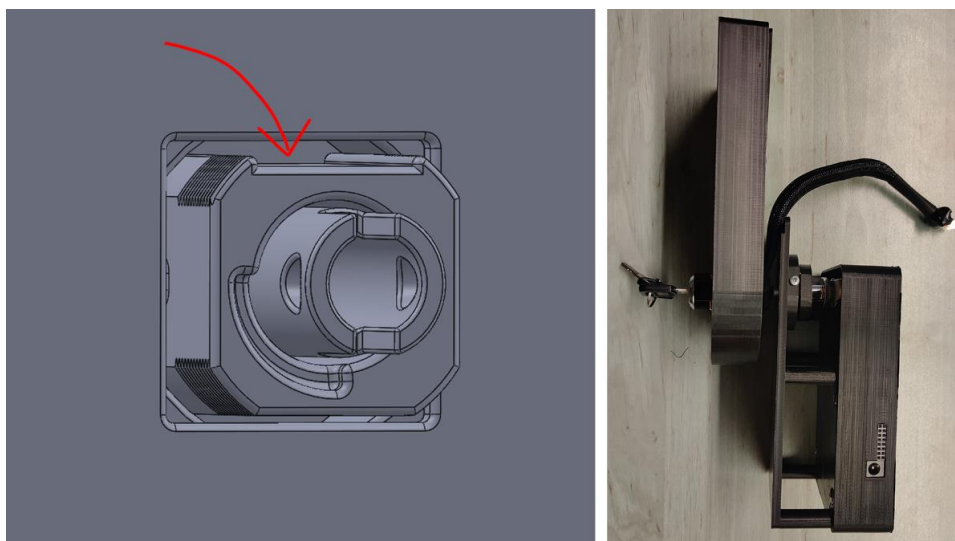
Targa laborikapi moodul on disainitud nii, et enne toote kliendile andmist, tuleb tootjal osad detailid kokku panna. Kliendi ülesandeks jääb aga toote edasine paigaldus kapile. Käesoleva peatüki esimene alapeatükk kirjeldab tootja vaatepunktist komponente, mis tuleb eelnevalt kokku panna ja kliendile anda ning teine alapeatükk kirjeldab kliendile antavaid juhiseid mooduli lõplikuks paigaldamiseks.

5.1 Kliendile antavad komponendid

Lõpliku tootena antakse kliendile kolm valmis komponenti: välimine korpus koos Gerd Kanteriga, kinnitusdetailid Elektroonikakarbi, kapi hoovasüsteemi ja Ekraanikorpuse omavaheliseks kinnitamiseks ning Elektroonikakarp.

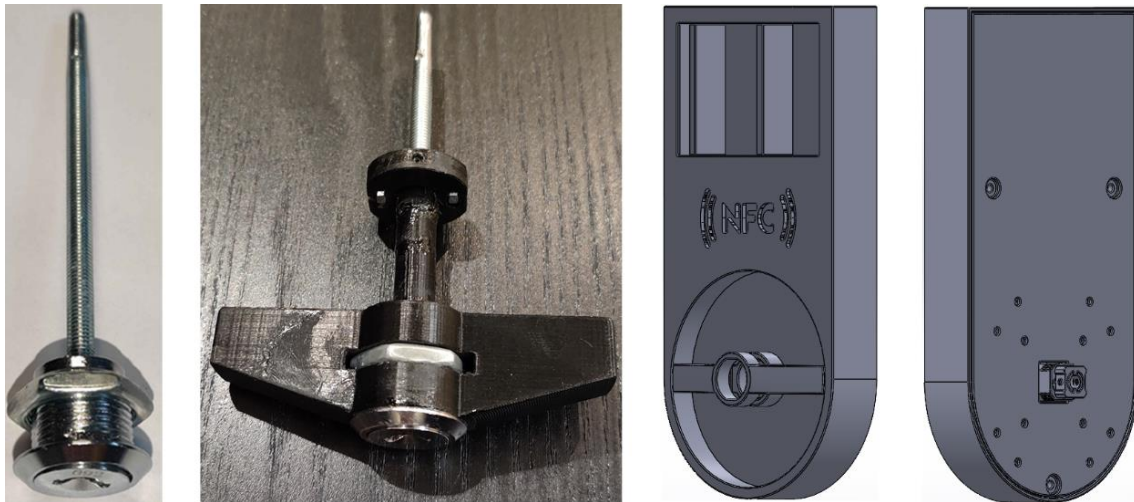
5.1.1 Välimine korpus koos Gerd Kanteriga

Ekraanikorpusesse peavad olema eelnevalt paigaldatud ekraanimoodul ning NFC moodul koos vajalike juhtmetega. Korpusel peab olema kaas kruvidega kinnitatud ning 15 viiguga juhe tuleb läbi kaane ja Kimalase vahel oleva ava. Kuna 15 viiguline pistik ise avast läbi ei mahu, tuleb juhe sealt läbi vedada enne kaane kinnitamist. Joonisel 40 on näha juhtmeava lõplikul moodulil ning prototüüpi koos juhtmega.



Joonis 40. Ava Ekraanikorpusest tulevate juhtmete jaoks ning prototüüp koos juhtmega.

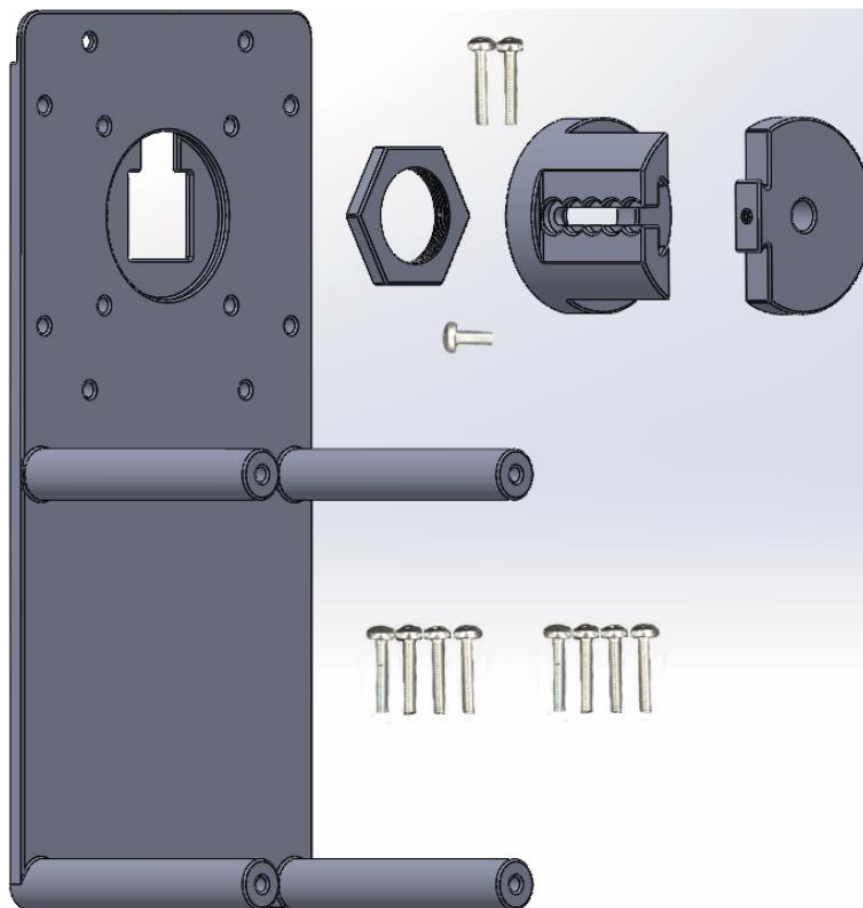
Lisaks peab Ekraanikorpusesse olema paigaldatud Kimalane kahe M3 suuruses kruvi või poldiga ning Käepide koos Toru, postkastilukuga ja sellest tuleva pika poldiga, kuid ilma poldi otsa käiva keeleta. Käepideme otsa on kinnitatud ka Gerd Kanter koos kahe M3 kruvi või poldiga, tänu millele ongi võimalik anda kliendile toode nii, et Käepide oleks kohe mooduli küljes. Joonisel 41 on näha prototüübiks kasutuses olnud käepidet koos postkastiluku, pika poldi ja Gerd Kanteriga ning mooduli välimist korpust koos Kaane, Kimalase ja Gerd Kanteriga.



Joonis 41. Prototüübiks kasutuses olnud Käepide ja esimese valmis komponendi jaoks vajalikud detailid.

5.1.2 Kinnitusdetailid

Kinnitusdetailidena antakse kliendile IKEA Laud, Hoidik, Naagelvärk, Mutter, üks ligikaudu 9 mm pikkune M3 kruvi või polt Naagelvärgi kinnitamiseks Hoidiku külge ja kümme M3x16 mm polti, millest kahte kasutatakse Hoidiku kinnitamiseks Gerd Kanteri külge, nelja Elektroonikakarbi kinnitamiseks IKEA Laua külge ning viimaseid keeratakse vajadusel Tuulerõugetesse. Need detailid ei pea olema kuidagi ette valmistatud, vaid neid kasutab klient nutikapi mooduli paigaldamiseks. Joonisel 42 on näha vajalikud kinnitusvahendid.



Joonis 42. Kinnitusvahendid targa laborikapi mooduli paigaldamiseks.

5.1.3 Elektroonikakarp

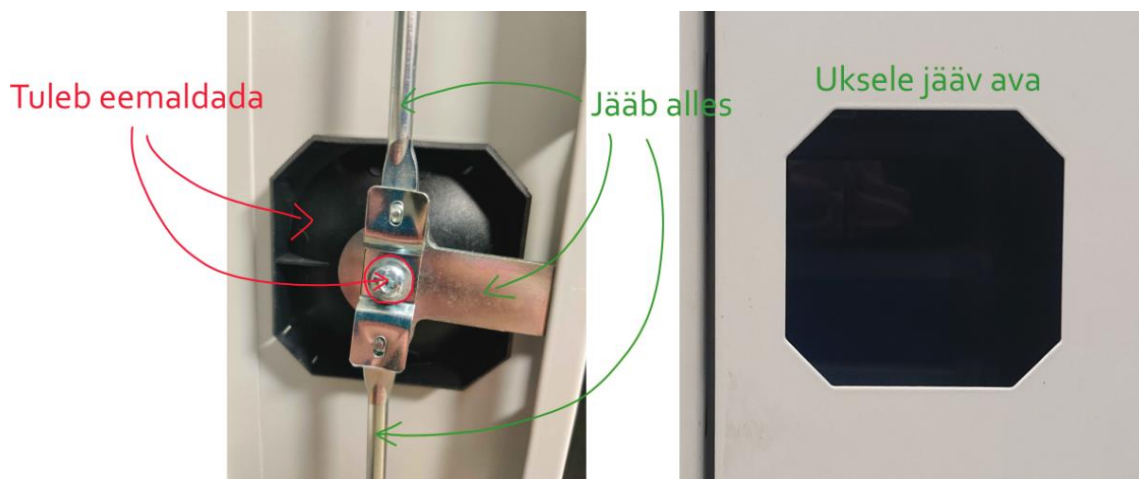
Elektroonikakarpi peab olema eelnevalt paigaldatud solenoid ja kogu mooduli tööks vajaminev elektroonika v.a ekraanimoodul ja NFC moodul. Solenoid peab olema kahe kruviga karbi küljes kinni, Elektroonikakarbi Kaas peab olema kinnitatud nelja M3 või M3.5 kruviga ning 12 V emane toitepistik ja 15 viiguga emane ekraani ning NFC pistik peavad olema toodud Elektroonikakarbi küljel oleva auguni. Joonisel 43 on näha modelleeritud detaile kolmanda valmis komponendi jaoks ja pistikuid Elektroonikakarbi küljel.



Joonis 43. Detailid kolmanda valmis komponendi jaoks ning pistikud Elektronikakarbi küljel.

5.2 Mooduli paigaldamine kapile

Esimene sammuna mooduli paigaldamisel tuleb ukselt küljest võtta käepide ning hoovasüsteem. Originaalsest süsteemist tuleb alles jätta kogu hoovasüsteem ja kapi ukse keel. Kuigi iga laborikapi ukse avamise mehhanismid on natuke erinevad, on ühte kappi näha joonisel 44.

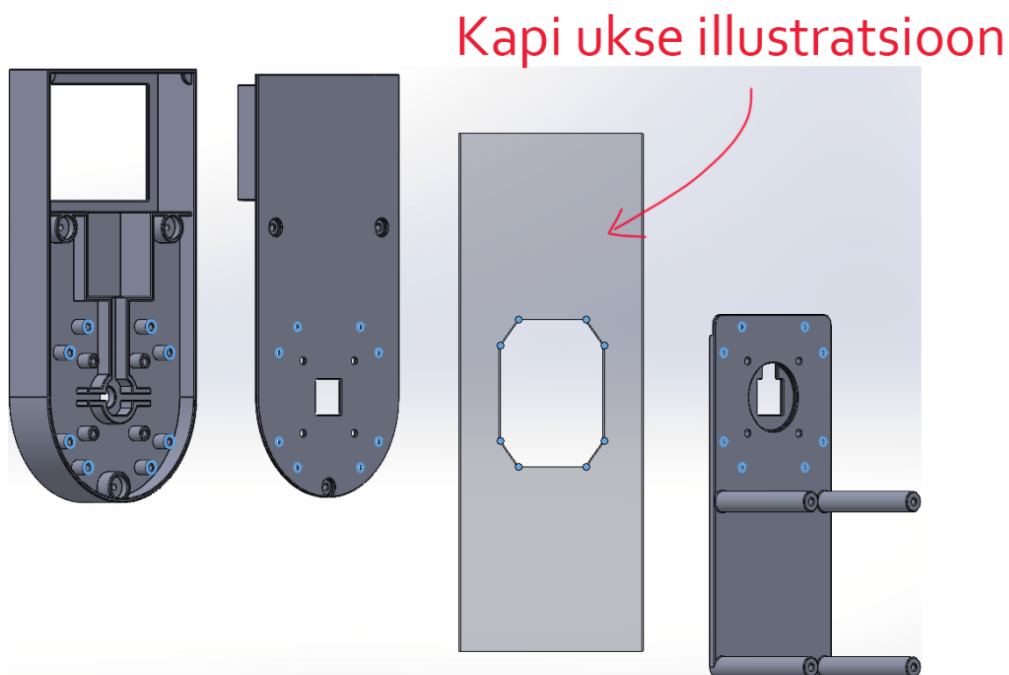


Joonis 44. Näidis laborikapist eemaldatavatest detailidest ja järele jääv ava.

Peale ebavajalike detailide eemaldamist jääb kapi uksel nähtavale ava, läbi mille oli originaalne käepide paigaldatud. Sellest august tuleb läbi panna Kimalane, mis on eelnevalt kinnitatud välimise korpuse külge. Laborikapi ustes olevad avad võivad kuju ja suuruse poolest märgatavalt erineda. Hetkel on nutimooduli Kimalase horisontaalne

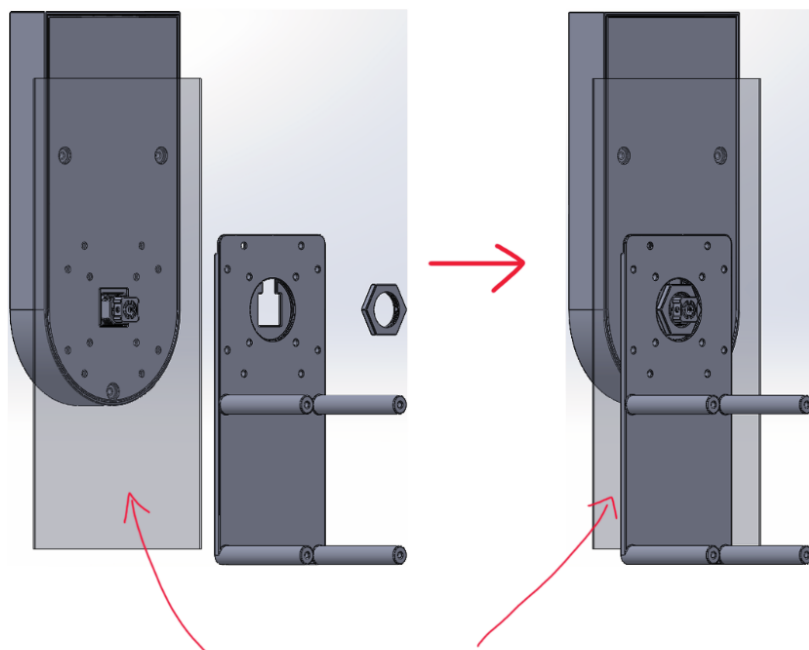
läbimõõt ligikaudu 16 mm ning ta sobitub ühe testitud laborikapi külge. Probleem tekkis aga joonisel 44 olevale laborikapile mooduli paigaldamisel, kuna ukse ava horisontaalne läbimõõt on üle 71 mm.

Selleks, et targa laborikapi avamise mehhanism toimiks, on oluline, et moodul oleks paigaldatud ukse ava keskpunkti, kuna muidu ei ulatu kapi originaalsed riivid üles ja alla serva ning ei saa ust kinni hoida. Probleemi lahenduseks modelleeriti Ekraanikorpusele, Ekraanikorpuse Kaanele ja IKEA Lauale Tuulerõuged. Nende abil on võimalik keerata kapi seest Ekraanikorpusesse M3x16 mm poldid, mis toimivad tugipunktidenä ning ei lase moodulil ukse küljes vertikaalselt ega horisontaalselt liikuda. Joonisel 45 on sinisega märgitud need Tuulerõuged, mis sobivad kokku joonisel 44 oleva laborikapi avaga. Ülejäänud neist aga võimaldavad moodulit paigaldada ka mõnele muud tüüpi laborikapile, mille peal prototüüpi pole testitud.



Joonis 45. Ühe testitud laborikapi jaoks vajaminevad Tuulerõuged mooduli stabiliseerimiseks.

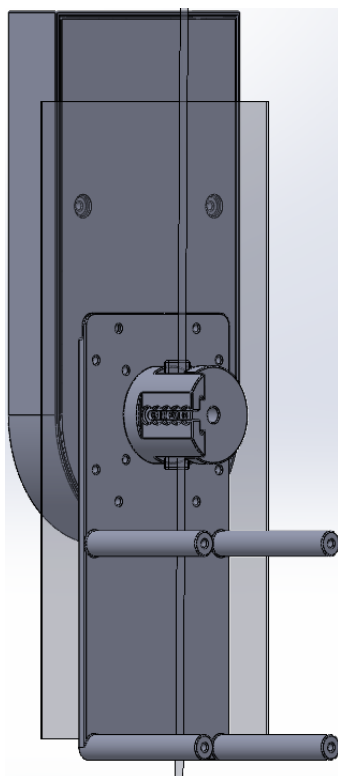
Teise sammuna tuleb panna Kimalase otsa uksest sissepoole IKEA Laud ning keerata see Mutriga tugevalt ukse külge. See tegevus on kujutatud joonisel 46.



Kapi ukse illustratsioon

Joonis 46. Välimise korpuse kinnitamine laborikapi ukse külge.

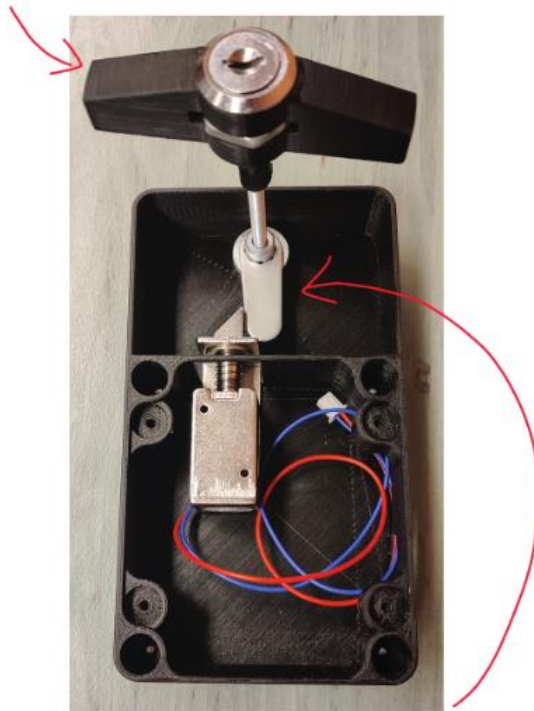
Kolmandaks kinnitub Gerd Kanteri otsa kahe M3x16 mm poldiga Hoidik ning Hoidiku otsa ligikaudu 9 mm pikkuse poldiga Naagelvärk (vt Joonis 47).



Joonis 47. Hoidiku ja Naagelvärgi kinnitamine Gerd Kanteri külge.

Neljandaks tuleb kinnitada Käepideme poldi süvendisse postkastiluku keele komponent ning suunata see Elektroonikakarbis asuvasse tugipunkti nii, et keel jääb allapoole (vt Joonis 48).

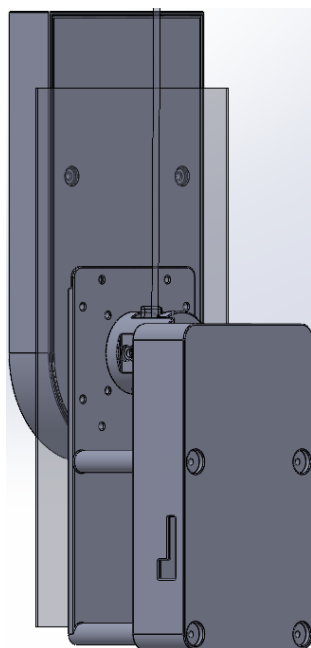
Käepide on horisontaalselt



Keel on suunatud alla

Joonis 48. Käepideme ja keele asendid Elektroonikakarbis.

Viies samm on Elektroonikakarbi keeramine nelja poldiga IKEA Laua külge ja juhtmete ühendamine karbi pistikutesse. Lõplikult paigaldatud targa laborikapi moodulit on näha joonisel 49.



Joonis 49. Lõplikult paigaldatud targa laborikapi moodul.

6 Edasiarenduse võimalused

Antud projekti saab edasi arendada mitmel erineval viisil. Projekti autoritel jäid töö käigus mõned olulised asjad tähelepanuta ja ajanappuse tõttu ei olnud kõike võimalik paremaks teha.

Kuna prototüüp on valdavas osas plastikust, siis järgmine samm olekski reaalselt mõned detailid metallist toota ja proovida neid katsetada mõnel laborikapil. Hetkel on polt, võllikrae, nelinurkne mutter, postkastiluku keel ja seib ostetud ehitusmaterjalide poest ning vastavalt vajadusele saetud, lihvitud ja kokku liimitud. Kõik need detailid võiksid olla aga modelleeritud ühe komponendina, et seda oleks lihtsam toota. Küll aga tuleks kindlasti üle veenduda, et mudelite mõõtmed on ikka korrektsed ning samuti tuleks uurida, kas praegustele lõpliku mooduli detailidele on keermed õigesti modelleeritud.

Riistvara koha pealt saaks suurema osa elektroonika jaoks trükkplaadi luua selle asemel, et joodetavale makettplaadile kõik joota. Trükkplaat on kompaktsem, töökindlam ja võtab ka vähem ruumi mooduli Elektroonikakarbi sees.

Selleks, et lahendus oleks rohkem nagu toode, tuleks tarkvara arendada edasi nii, et Wi-Fi ühendust, Firebase'i andmebaasi ühendusstringe ja API võtmeid kui ka kapi kasutajakontot saaks lisada läbi ekraani. Töös saadud lahendus on *hardcoded* ja eelnevalt nimetatud tegevusi ei saa teha.

7 Kokkuvõte

Töö eesmärk oli ehitada võimalikult kompaktne, soodne, universaalne ja lihtsasti paigaldatav moodul, mis muudaks metallist laborikapi targaks. Tarkvara pidi võimaldama ka mitme kapi kasutust. Lahendus pidi kasutama ka kapi originaalset riivi ja voolu kadumisel pidi kapp olema avatav ka võtmega. Eesmärk oli muuta ka kapp jälle kaardiga avatavaks.

Peaaegu kõik projektile seatud eesmärgid saadi täidetud ning lõpuks valmis kompaktne ühe mooduliga piirduv süsteem, mis ei ole kuigi kulukas ja on suhteliselt universaalne ehk see on paigaldatav erinevatele Tallinna Tehnikaülikooli laborikappidele. Tänu mooduli kompaktsusele on see ka kergesti paigaldatav. Veebirakendus toetab mitme kapi kasutust ning eelmise projekti probleem, et kappi ei saanud enam ühel hetkel kaardiga avada, on samuti lahendatud. Lõplik moodul kasutab kapi algset riivi ja ühtlasi on seda võimalik avada võtmega. Küll aga loodud laborikapp ei ole autorite definitsiooni järgi tark. Autorid on defineerinud targa laborikapi kapina, mida on võimalik avada ISIC kaardiga ning kuhu saab lisada kasutajaid mooduli ekraani kaudu ja läbi internetirakenduse. Praegusel juhul aga lõppmoodulil ekraani testitud ei ole, seega laborikappi, millele moodul on paigaldatud, targaks nimetada ei saa.

Kasutatud kirjandus

- [1] M. Visnapuu, R. Randmäe, S. Tišler ja M. Veersoo, „Nutikas laborikapp,“ GitHub, 2023. [Vörgumaterjal]. Available: <https://github.com/siimtishler/kapp>. [Kasutatud september 2024].
- [2] V. Abrol, „digitrix,“ 4 oktoober 2024. [Vörgumaterjal]. Available: <https://www.digitrix.com/blogs/nfc-why-nfc-is-popular-and-why-everyone-tries-to-use-it>. [Kasutatud detsember 2024].
- [3] S. Maestre, „Circuitbread,“ 17 juuni 2021. [Vörgumaterjal]. Available: <https://www.circuitbread.com/ee-faq/how-does-a-flyback-diode-work>. [Kasutatud november 2024].
- [4] Marla, „electronics.stackexchange,“ 15 Mai 2014. [Vörgumaterjal]. Available: <https://electronics.stackexchange.com/questions/110574/how-to-choose-a-flyback-diode-for-a-relay>. [Kasutatud november 2024].
- [5] Diodes Incorporated, „alldatasheet,“ [Vörgumaterjal]. Available: <https://www.alldatasheet.com/datasheet-pdf/view/59328/DIODES/SR303.html>. [Kasutatud november 2024].
- [6] „einstronic,“ [Vörgumaterjal]. Available: <https://einstronic.com/product/1-channel-5v-relay-module-ky-019/>. [Kasutatud november 2024].
- [7] Admin, „how2electronics,“ 29 mai 2023. [Vörgumaterjal]. Available: <https://how2electronics.com/interfacing-pn532-nfc-rfid-module-with-arduino/>. [Kasutatud oktoober 2024].
- [8] G. Schorcht, „doc.riot-os,“ [Vörgumaterjal]. Available: https://doc.riot-os.org/group__boards__esp32__mh-et-live-minikit.html. [Kasutatud oktoober 2024].
- [9] M.-W. lady ada, „learn.adafruit,“ 19 august 2014. [Vörgumaterjal]. Available: <https://learn.adafruit.com/adafruit-3-5-color-320x480-tft-touchscreen-breakout/spi-wiring-and-test#spi-mode-jumpers-5-3>. [Kasutatud detsember 2024].
- [10] Ava, „Unveiling the Difference Between ESP32 and ESP8266,“ Ersaelectronics, [Vörgumaterjal]. Available: <https://www.ersaelectronics.com/blog/unveiling-the-difference-between-esp32-and-esp8266>.
- [11] Google, „Developer documentation for Firebase,“ Firebase, 2024. [Vörgumaterjal]. Available: <https://firebase.google.com/docs>. [Kasutatud november 2024].
- [12] ChatGPT-4o. (2024), OpenAI, [Vörgumaterjal]. Available: <https://chat.openai.com>. [Kasutatud november 2024].
- [13] E. You, „Vue.js Guide,“ Vue.js, 2024. [Vörgumaterjal]. Available: <https://vuejs.org/guide>. [Kasutatud november 2024].
- [14] A. Kool, „Firebase kicks out current user,“ Stack Overflow, 24 juuni 2016. [Vörgumaterjal]. Available:

- <https://stackoverflow.com/questions/37517208/firebase-kicks-out-current-user/38013551#38013551>. [Kasutatud november 2024].
- [15] Bodmer, „TFT_eSPI“, GitHub, 2024. [Võrgumaterjal]. Available: https://github.com/Bodmer/TFT_eSPI. [Kasutatud november 2024].
- [16] Adafruit, „3.5" TFT 320x480 + Touchscreen Breakout Board w/MicroSD Socket“, Adafruit, 2024. [Võrgumaterjal]. Available: <https://www.adafruit.com/product/2050>. [Kasutatud november 2024].
- [17] Amazon Web Services, „FreeRTOS Kernel Developer Docs“, FreeRTOS, 2024. [Võrgumaterjal]. Available: <https://freertos.org/Documentation/02-Kernel/02-Kernel-features/00-Developer-docs>. [Kasutatud november 2024].

Lisa 1 – Link koodihoidlale ja veebilehele

- Koodihoidla GitHubis: <https://github.com/krikaa/kappV2>
- Demoveebileht: <https://nutikapp-v2.web.app>
 - Kasutajanimi: demo, parool: nutikapp

Lisa 2 – Reaalaja andmebaasi turvareeglid

```
{
  "rules": {
    "lockers": {
      ".read": "auth != null &&
        root.child('users').child(auth.uid)
          .child('role').val() === 'admin'",
      ".write": "auth != null &&
        root.child('users').child(auth.uid).
          child('role').val() === 'admin'",
      "$lockerName": {
        ".read": "auth != null &&
          root.child('users').child(auth.uid).child('access')
            .child($lockerName).val() === true",
        ".write": "auth != null &&
          root.child('users').child(auth.uid).child('access')
            .child($lockerName).val() === true",
      }
    },
    "users": {
      ".read": "auth != null &&
        root.child('users').child(auth.uid)
          .child('role').val() === 'admin'",
      ".write": "auth != null &&
        root.child('users').child(auth.uid)
          .child('role').val() === 'admin'",
      "$user_id": {
        ".read": "auth != null && auth.uid === $user_id",
      }
    }
  }
}
```